

Async remote inference Code walkthrough

Fully expanded reading edition

All 26 slides, explanations, review questions and answers, and every embedded source-code tab. This edition is self-contained: repository links are not needed to read the included code.

The interactive browser presentation is the source. Its two experiments are represented here by their default illustrations and additional static cases. PDF controls do not run inference or simulate a robot.

Use the contents or PDF bookmarks to move between chapters. Each lesson starts with its visual slide and explanation, followed by the expanded source excerpts. Wrapped code lines retain the original line number on the first line.

Source snapshot: c73a3f0403046f669326ca896755b00cbef415fa (30 September 2026). Exported 1 October 2026. The original excerpts remain intentionally bounded; this PDF includes all of them, not the entire repository.

The material describes implemented behavior and distinguishes it from deferred validation. It is a learning companion, not a completed code review.

Contents

1 · Map the system	4
01. Async remote inference: the code	4
02. Ownership follows the process boundary	7
03. Two startup paths meet at admission	10
04. The canonical data boundary	13
05. Different identifiers prevent different mistakes	16
2 · Trace one request	19
06. Admission establishes a complete contract	19
07. A control tick never waits for inference	22
08. Concurrency is deliberately bounded	25
09. The request crosses a bounded transport	28
10. PolicyRunner is the model boundary	31
11. Receiving a result does not authorize motion	34
3 · Read execution	37
12. The cursor counts committed endpoints	37
13. Append and alignment place a chunk differently	39
14. Blending operates on matching future slots	41
15. Refill is a playback threshold	44
16. Explore request timing	47
17. RTC changes the prediction itself	50
4 · Review lifecycle	53
18. Runtime deadlines and presence grace	53
19. Retargeting and reset are different operations	55

20. Session cleanup preserves model ownership	57
21. Terminal fault becomes local shutdown	60
22. Language is a planned hold path	63
5 · Prepare your review	66
23. Policy compatibility is an execution contract	66
24. Read logs as a causal chain	69
25. Tests encode the important counterexamples	72
26. A focused first code review	74

1 · Map the system

01. Async remote inference: the code

1 · MAP THE SYSTEM

Async remote inference: the code

A guided route from your hardware intuition to a code review.

01**Map ownership**

Who can change
which state?

02**Trace one
request**

Capture → prediction
→ dispatch

03**Read execution**

Timing, alignment and
blending

04**Review
boundaries**

Invalidation, faults and
tests

Start with one chapter. Open code only when you want the next level of detail.

01

This companion describes the checked-out implementation on 30 September 2026. It includes the aligned playback-threshold correction and the later lifecycle changes. The earlier high-level deck is historical and describes that correction as planned.

Use the slide to form a mental model, then open “Explain & code” for the reasoning and exact source excerpts. Try the question before revealing its answer. The two interactive examples change explanatory numbers only; they never connect to a robot or server.

This is a learning guide, not a completed code review or a new specification. Hardware task/motion and focused lifecycle checks have operator-reported acceptance. Real language-model behavior and representative routed/WAN deployment remain separate work.

Check your understanding

Which part should remain responsive while inference is running?

Answer

The local robot control loop. Network waits and policy execution live outside it.

01. Source excerpts - all tabs expanded

1. RemoteInferenceEngine

src/lerobot/rollout/inference/remote.py:36

Excerpt lines 36-67 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

36 class RemoteInferenceEngine(InferenceEngine):
37     def __init__(
38         self,
39         client: RemoteClient,
40         config: RemoteInferenceConfig,
41         dataset_features: dict,
42         rename_map: dict[str, str],
43         robot_wrapper: ThreadSafeRobot,
44         task: str,
45         shutdown_event: Event | None = None,
46     ) -> None:
47         super().__init__(task)
48         self.client = client
49         self.config = config
50         self._features = dataset_features
51         self._rename_map = rename_map
52         self._robot = robot_wrapper
53         self._global_shutdown = shutdown_event
54         self.runtime = ChunkRuntime(
55             mode=ExecutionMode(config.mode),
56             action_interval=client.capabilities.action_interval,
57             refill_seconds=config.refill_seconds,
58             max_observation_age_s=config.max_observation_age_s,
59             action_timeout_s=config.action_timeout_s,
60             startup_timeout_s=config.startup_timeout_s,
61             training_max_delay=client.capabilities.training_max_delay,
62             chunk_merge=config.chunk_merge,
63             blend_steps=config.blend_steps,
64             blend_weight=config.blend_weight,
65             blend_indices=tuple(
66                 client.capabilities.action_feature.names.index(name) for name in config.blend_components
67             ),

```

2. ChunkRuntime

src/lerobot/inference/execution.py:62

Excerpt lines 62-93 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

62 class ChunkRuntime:
63     """Capacity-one executor scheduling with per-action provenance and latched faults."""
64
65     def __init__(
66         self,
67         *,
68         mode: ExecutionMode,
69         action_interval: float,
70         refill_seconds: float,
71         max_observation_age_s: float,
72         action_timeout_s: float,
73         startup_timeout_s: float,
74         training_max_delay: int = 0,
75         chunk_merge: str = "append",
76         blend_steps: int = 0,
77         blend_weight: float = 0.5,
78         blend_indices: tuple[int, ...] = (),
79         clock: Callable[[], float] = time.monotonic,
80     ) -> None:
81         """Allocate one action queue and a bounded steady-state latency window."""
82         for value in (action_interval, max_observation_age_s, action_timeout_s, startup_time

```

```
    out_s):
83         if not math.isfinite(value) or value <= 0:
84             raise ValueError("Execution intervals, ages and deadlines must be finite and
    positive")
85         if not math.isfinite(refill_seconds) or refill_seconds < 0:
86             raise ValueError("Refill playback threshold must be finite and nonnegative")
87         if chunk_merge not in {"append", "aligned"} or (
88             chunk_merge != "append" and mode is not ExecutionMode.CHUNK
89         ):
90             raise ValueError("Aligned replacement is supported only for plain chunks")
91         if type(blend_steps) is not int or blend_steps < 0:
92             raise ValueError("Blend steps must be a nonnegative integer")
93         if not math.isfinite(blend_weight) or not 0 < blend_weight <= 1:
```

1 · Map the system

02. Ownership follows the process boundary

1 · MAP THE SYSTEM

Ownership follows the process boundary

The client owns motion permission. The server owns policy execution.

Robot client

Rollout strategy → **RemoteInferenceEngine** → **ChunkRuntime** → **ActionQueue**

Reads hardware, commits targets, interpolates, checks freshness and faults.

GPU server

PolicyServer → **SessionWorker** → **PolicyRunner** → **policy + processors**

Admits one session and serializes every operation that can mutate model state.

The shared runtime decides how to execute a result; Zenoh only carries the request and reply.

02

RemoteInferenceEngine coordinates a background network worker with the local control loop. ChunkRuntime owns scheduling, acceptance and dispatch permission. ActionQueue stores paired action representations with provenance and a commitment cursor.

SessionWorker owns exclusive policy/processor access. PolicyRunner knows canonical observations, preprocessing and model calls, but knows nothing about robot I/O or Zenoh. Shared execution concepts also support local RTC; synchronous select_action behavior remains its own path.

The review question is about authority: a network callback cannot send a motor command, and the control thread cannot reset the server policy while a forward call is in progress.

Check your understanding

Where would a change to plain alignment belong?

Answer

In shared execution and its queue contract, with negotiation as needed—not in transport callbacks or a particular model forward method.

02. Source excerpts - all tabs expanded

1. RemoteInferenceEngine._loop

src/lerobot/rollout/inference/remote.py:374

Excerpt lines 374-405 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

374     def _loop(self) -> None:
375         try:
376             while not self._stop_event.is_set() and not self.failed:
377                 # Control-thread dispatch only queues bounded diagnostics. Log
378                 # from this worker so first-dispatch reporting adds no motor I/O.
379                 with self.runtime.lock:
380                     dispatch_events = list(self.runtime.dispatch_events)
381                     self.runtime.dispatch_events.clear()
382                 for event in dispatch_events:
383                     self._event("first_dispatch", **event)
384                 with self._lock:
385                     control = self._control
386                     observation = self._observation
387                     hold_ready = self._hold_requested and self._hold_acknowledged
388                 if control is not None:
389                     # Language invalidation must be acknowledged by the motor thread
390                     # before submitting any server-side generation operation.
391                     if self._hold_requested and not hold_ready:
392                         self._stop_event.wait(0.002)
393                         continue
394                     operation, generation = control
395                     self.client.control(operation, generation)
396                     with self._lock:
397                         if self._control == control:
398                             self._control = None
399                         continue
400                 if not self.runtime.active or observation is None:
401                     self.runtime.check_deadlines()
402                     self._stop_event.wait(0.002)
403                     continue
404                 if hold_ready:
405                     with self._lock:

```

2. SessionWorker._run

src/lerobot/remote_inference/server.py:537

Excerpt lines 537-568 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

537     def _run(self) -> None:
538         while not self._stopping.is_set():
539             try:
540                 message, future, queued_at = self._commands.get(timeout=0.05)
541             except Empty:
542                 continue
543             started = time.monotonic()
544             inference_operation = message.message_type in {
545                 MessageType.OBSERVATION,
546                 MessageType.LANGUAGE_REQUEST,
547             }
548             deadline = (
549                 self.language_deadline_s
550                 if message.message_type is MessageType.LANGUAGE_REQUEST
551                 else self.action_deadline_s
552             )
553             try:
554                 if inference_operation and started - queued_at > deadline:
555                     response = message.error(ErrorCode.TIMEOUT, "Policy worker queue deadlin
e exceeded")
556                 if self._session is not None:
557                     self._session.faulted = True

```

```

558         else:
559             response = self._execute(message)
560     except Exception as exc:
561         logger.exception(
562             "Policy worker failure instance=%s session=%s request=%s",
563             self.instance_id,
564             message.session_id,
565             message.request_id,
566         )
567         timed_out = time.monotonic() - queued_at > deadline
568         response = message.error(ErrorCode.TIMEOUT if timed_out else ErrorCode.EXECU
TION, str(exc))

```

3. PolicyRunner.predict

src/lerobot/inference/policy_runner.py:217

Excerpt lines 217-248 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

217     def predict(
218         self,
219         observation: ObservationSnapshot,
220         *,
221         mode: ExecutionMode = ExecutionMode.CHUNK,
222         inference_delay: int = 0,
223         model_continuation: torch.Tensor | None = None,
224         canonical_continuation: torch.Tensor | None = None,
225         provenance: ActionProvenance | None = None,
226     ) -> ActionChunk:
227         """Preprocess a fresh snapshot and return a validated canonical action chunk."""
228         mode = ExecutionMode(mode)
229         if mode not in self.capabilities.modes:
230             raise ValueError(f"Execution mode {mode.value!r} was not enabled by this deploym
ent.")
231         started = time.perf_counter()
232         # Guided RTC locally enables autograd for its prefix correction. no_grad
233         # permits that override; inference_mode would suppress its gradient graph.
234         with torch.inference_mode(mode is not ExecutionMode.RTC_GUIDED), torch.no_grad():
235             prepared = self.preprocessor(self._batch(observation))
236             preprocessed_at = time.perf_counter()
237             kwargs: dict[str, Any] = {}
238             if mode != ExecutionMode.CHUNK:
239                 prefix = self._prepare_continuation(model_continuation, canonical_continuati
on)
240             if inference_delay < 0:
241                 raise ValueError("RTC delay must be nonnegative.")
242             if prefix is None and inference_delay:
243                 raise ValueError("RTC delay requires a real continuation prefix.")
244             if mode == ExecutionMode.RTC_TRAINED:
245                 available = 0 if model_continuation is None else len(model_continuation)
246                 if inference_delay > min(
247                     self.capabilities.training_max_delay, available, self.capabilities.r
tc_horizon
248                 ):

```

1 · Map the system

03. Two startup paths meet at admission

1 · MAP THE SYSTEM

Two startup paths meet at admission

Weights stay on the server; the client builds a hardware contract.

Server startup

load_deployment → **PolicyRunner** → **warmup** → **full reset** → **serve**

Checkpoint + saved processors + effective settings become one advertised deployment.

Client startup

connect / describe → **robot schema** → **admit** → **RemoteInferenceEngine**

Camera names, joint order, semantics, action rate and mode must agree.

Successful warmup proves a callable path; it does not prove task performance.

03

`load_deployment` resolves the checkpoint (and PEFT base if configured), loads the policy and canonical processors, constructs the runner, computes artifact identity and warms the enabled paths. A full reset removes warmup state before readiness.

`build_remote_rollout_context` explicitly rejects a local policy, device or compilation setting. It discovers capabilities, connects hardware, collects the initial pose, aligns component order, applies the rename map and requests admission. Hardware connection and observation happen during setup; policy dispatch waits for compatibility and admission.

The artifact hash identifies loaded files and effective configuration. It is not a hash of all implementation code; package/revision information is logged separately.

Check your understanding

Does “compatible before motion” mean the robot was never connected?

Answer

No. Setup connects and reads the robot to construct the contract. The guarantee concerns admitting remote policy execution and dispatching its actions.

03. Source excerpts - all tabs expanded

1. load_deployment

src/lerobot/scripts/lerobot_policy_server.py:65

Excerpt lines 65-96 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

65 def load_deployment(cfg: ServerConfig) -> tuple[PolicyRunner, str]:
66     """Resolve contents, load canonical processors and reset all warmed model paths."""
67     if cfg.action_feature is None:
68         raise ValueError("An explicit action feature is required")
69     path = resolve_artifact(cfg.model.repo_or_path, cfg.model.revision)
70     artifacts = {"checkpoint": path}
71     policy_cfg = PreTrainedConfig.from_pretrained(path)
72     policy_cfg.pretrained_path = path
73     policy_cfg.device = cfg.model.device
74     if hasattr(policy_cfg, "rtc_config"):
75         policy_cfg.rtc_config = cfg.execution.rtc
76     policy_class = get_policy_class(policy_cfg.type)
77     if policy_cfg.use_peft:
78         require_package("peft", extra="peft")
79         adapter = PeftConfig.from_pretrained(str(path))
80         base = resolve_artifact(adapter.base_model_name_or_path, adapter.revision)
81         artifacts["base"] = base
82         policy = policy_class.from_pretrained(base, config=policy_cfg)
83         policy = cast(PreTrainedPolicy, PeftModel.from_pretrained(policy, path, config=adapt
er))
84     else:
85         policy = policy_class.from_pretrained(path, config=policy_cfg)
86         policy.to(cfg.model.device).eval()
87         preprocessor, postprocessor = make_pre_post_processors(
88             policy_cfg=policy_cfg,
89             pretrained_path=path,
90             preprocessor_overrides={
91                 "device_processor": {"device": cfg.model.device},
92                 "rename_observations_processor": {"rename_map": {}},
93             },
94         )
95         runner = PolicyRunner(
96             policy,

```

2. build_remote_rollout_context

src/lerobot/rollout/remote_context.py:51

Excerpt lines 51-82 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

51 def build_remote_rollout_context(
52     cfg: RolloutConfig,
53     shutdown_event: Event,
54     teleop_action_processor: RobotProcessorPipeline | None = None,
55     robot_action_processor: RobotProcessorPipeline | None = None,
56     robot_observation_processor: RobotProcessorPipeline | None = None,
57 ) -> RolloutContext:
58     config = cfg.inference
59     if not isinstance(config, RemoteInferenceConfig) or cfg.robot is None:
60         raise ValueError("Remote rollout requires remote inference and robot configuration")
61     if cfg.policy is not None or cfg.device is not None or cfg.use_torch_compile:
62         raise ValueError("Remote rollout does not accept local policy/device/compile configu
ration")
63     client = RemoteClient.connect(config)
64     robot = None
65     teleop = None
66     try:
67         descriptor = client.descriptor
68         capabilities = descriptor["capabilities"]
69         expected = tuple(FeatureSpec(**feature) for feature in capabilities["features"])
70         action = FeatureSpec(**capabilities["action_feature"])

```

```

71         if not action.names:
72             raise ValueError("Remote rollout requires explicit ordered canonical action comp
onent names")
73
74         default_teleop, default_action, default_observation = make_default_processors()
75         teleop_action_processor = teleop_action_processor or default_teleop
76         robot_action_processor = robot_action_processor or default_action
77         robot_observation_processor = robot_observation_processor or default_observation
78
79         robot = make_robot_from_config(cfg.robot)
80         wrapper = ThreadSafeRobot(robot)
81         wrapper.configure_position_hold()
82         robot.connect()

```

3. rollout

src/lerobot/scripts/lerobot_rollout.py:227

Excerpt lines 227-258 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

227 def rollout(cfg: RolloutConfig):
228     """Main entry point for policy deployment."""
229     init_logging()
230
231     if cfg.display_data:
232         logger.info(
233             "Initializing %s visualization (ip=%s, port=%s)",
234             cfg.display_mode,
235             cfg.display_ip,
236             cfg.display_port,
237         )
238         init_visualization(cfg.display_mode, session_name="rollout", ip=cfg.display_ip, port
=cfg.display_port)
239
240         signal_handler = ProcessSignalHandler(use_threads=True, display_pid=False)
241         shutdown_event = signal_handler.shutdown_event
242         if not isinstance(shutdown_event, threading.Event):
243             raise RuntimeError("ProcessSignalHandler(use_threads=True) must hand out a threading
.Event")
244         if cfg.interactive:
245             # /reset and /stop end the control loop via the local flag; process signals still
246             # propagate through the parent event.
247             shutdown_event = LinkedEvent(shutdown_event)
248
249         logger.info("Building rollout context...")
250         ctx = build_rollout_context(cfg, shutdown_event)
251
252         strategy = create_strategy(cfg.strategy)
253         logger.info("Rollout strategy: %s", cfg.strategy.type)
254         logger.info(
255             "Robot: %s | FPS: %.0f | Duration: %s",
256             cfg.robot.type if cfg.robot else "?",
257             cfg.fps,
258             f"{cfg.duration}s" if cfg.duration > 0 else "infinite",

```

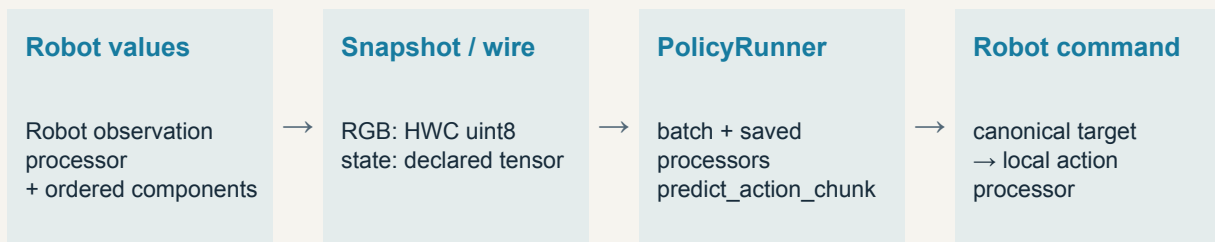
1 · Map the system

04. The canonical data boundary

1 · MAP THE SYSTEM

The canonical data boundary

The wire carries named observations and canonical action targets.



A six-element tensor is compatible only if its names, order and meaning also match.

04

FeatureSpec includes name, shape, dtype, kind, ordered component names and explicit semantic conventions. A shape match alone cannot tell degrees from normalized joint values or identify a swapped camera.

The client renames observation keys once. ObservationSnapshot copies arrays and marks them read-only because camera implementations may reuse buffers. On the server, `_batch` validates the exact negotiated schema, converts RGB HWC uint8 into batched CHW float values in `[0,1]`, then invokes the saved policy preprocessor.

Canonical actions are policy postprocessor outputs before the local robot action processor. Do not confuse model-normalized actions, canonical targets, post-processor commands and measured positions. They are deliberately separate in the event log.

Check your understanding

Why copy image arrays before asynchronous encoding?

Answer

The camera may overwrite a shared buffer while another thread is encoding it. The snapshot must own stable bytes.

04. Source excerpts - all tabs expanded

1. ObservationSnapshot

src/lerobot/inference/contracts.py:114

Excerpt lines 114-139. Line numbers refer to the documented source snapshot.

```

114 class ObservationSnapshot:
115     """Owned immutable current observations with original capture provenance."""
116
117     features: Mapping[str, np.ndarray]
118     capture_time: float
119     task: str
120     task_version: int = 0
121     observation_id: str = ""
122     action_cursor: int | None = None
123     execution_generation: int | None = None
124
125     def __post_init__(self) -> None:
126         """Copy camera buffers so asynchronous encoding sees a stable frame."""
127         if not math.isfinite(self.capture_time):
128             raise ValueError("Observation capture time must be finite.")
129         for anchor in (self.action_cursor, self.execution_generation):
130             if anchor is not None and (type(anchor) is not int or anchor < 0):
131                 raise ValueError("Observation action anchors must be nonnegative integers.")
132         # Cameras commonly recycle their arrays. Own exactly one bounded request
133         # snapshot; copies remain read-only until the runner creates its batch.
134         owned: dict[str, np.ndarray] = {}
135         for name, value in self.features.items():
136             array = np.array(value, copy=True, order="C")
137             array.setflags(write=False)
138             owned[name] = array
139         object.__setattr__(self, "features", MappingProxyType(owned))

```

2. PolicyRunner._batch

src/lerobot/inference/policy_runner.py:195

Excerpt lines 195-215. Line numbers refer to the documented source snapshot.

```

195     def _batch(self, observation: ObservationSnapshot) -> dict[str, Any]:
196         if set(observation.features) != {feature.name for feature in self.capabilities.features}:
197             raise ValueError("Observation feature names differ from the serving contract.")
198         if len(observation.task) > self.max_text_input:
199             raise ValueError("Instruction exceeds the configured text-input limit.")
200         batch: dict[str, Any] = {}
201         for feature in self.capabilities.features:
202             array = observation.features[feature.name]
203             if array.shape != feature.shape or array.dtype != np.dtype(feature.dtype):
204                 raise ValueError(
205                     f"Observation shape/dtype differs from the serving contract: {feature.name}.")
206             if array.dtype.kind == "f" and not np.isfinite(array).all():
207                 raise ValueError(f"Observation contains non-finite values: {feature.name}.")
208             tensor = torch.from_numpy(array.copy())
209             if feature.kind == "rgb":
210                 tensor = tensor.permute(2, 0, 1).contiguous().to(torch.float32).div_(255)
211             batch[feature.name] = tensor.unsqueeze(0)
212         batch["task"] = [observation.task]
213         batch["robot_type"] = self.robot_type
214         return batch
215

```

3. FeatureSpec

src/lerobot/inference/contracts.py:34

Excerpt lines 34-61. Line numbers refer to the documented source snapshot.

```

34 class FeatureSpec:
35     """Named canonical values, ordered components, and semantic conventions."""
36
37     name: str
38     shape: tuple[int, ...]
39     dtype: str
40     kind: str = "tensor"
41     names: tuple[str, ...] = ()
42     semantics: str = ""
43
44     def __post_init__(self) -> None:
45         """Reject unsupported modalities and ambiguous feature metadata."""
46         object.__setattr__(self, "shape", tuple(self.shape))
47         object.__setattr__(self, "names", tuple(self.names))
48         if not self.name or not self.semantics.strip():
49             raise ValueError("Feature names and explicit semantic conventions are required."
50 )
51         if self.kind not in {"tensor", "rgb"}:
52             raise ValueError(f"Unsupported feature modality: {self.kind!r}")
53         if len(self.shape) > 4 or any(type(dim) is not int or dim <= 0 for dim in self.shape
54 ):
55             raise ValueError(f"Invalid feature shape: {self.shape!r}")
56         if self.dtype not in {"bool", "uint8", "int32", "int64", "float32", "float64"}:
57             raise ValueError(f"Unsupported feature dtype: {self.dtype!r}")
58         if self.kind == "rgb" and (len(self.shape) != 3 or self.shape[-1] != 3 or self.dtype
59 != "uint8"):
60             raise ValueError("RGB features require HWC uint8 with three channels.")
61         if self.names and (len(self.shape) != 1 or len(self.names) != self.shape[0]):
62             raise ValueError("Ordered component names must match a one-dimensional feature."
63 )
64         if len(set(self.names)) != len(self.names):
65             raise ValueError("Feature component names must be unique.")

```

1 · Map the system

05. Different identifiers prevent different mistakes

1 · MAP THE SYSTEM

Different identifiers prevent different mistakes

A sequence number does not replace session identity or freshness.

instance + session	Which server process and exclusive client owner?
generation + task_version	Is this motion epoch and instruction still current?
request_id + sequence	Which operation, and has it already been consumed?
observation_id + cursor	Which captured state and committed action position?

Client monotonic time answers “how old?”; the action cursor answers “how much was committed?”

05

The server instance changes on restart. A session identifies exclusive ownership. Generation fences reset/invalidation boundaries; task_version distinguishes retargets, including $A \rightarrow B \rightarrow A$; setting the already-current text is a no-op. Request IDs correlate replies, while a monotonic data sequence prevents re-execution after bounded request-ID history is evicted.

The observation action_cursor is local scheduling metadata. It is separate from the wire request sequence. It counts policy endpoints already handed to interpolation, not encoder measurements or server time.

Client capture times are carried through the response as opaque provenance on the server. The client evaluates age with its own monotonic clock; the server reports local durations. Synchronized wall clocks are not required.

Check your understanding

Can identical task text make an old in-flight answer current again?

Answer

No. Version/generation/query intent checks still distinguish it from the new request.

05. Source excerpts - all tabs expanded

1. Envelope

src/lerobot/remote_inference/protocol.py:88

Excerpt lines 88-119 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

88 class Envelope:
89     """Every data message has a correlation identity; negotiation permits empty instance/session IDs."""
90
91     message_type: MessageType | str
92     instance_id: str = ""
93     session_id: str = ""
94     generation: int = 0
95     request_id: str = ""
96     body: dict[str, Any] = field(default_factory=dict)
97     version: int = PROTOCOL_VERSION
98
99     def __post_init__(self) -> None:
100         """Validate correlation fields before decoding potentially large bodies."""
101         if type(self.version) is not int or self.version != PROTOCOL_VERSION:
102             raise ProtocolError(ErrorCode.PROTOCOL, f"Unsupported protocol version: {self.version!r}")
103         try:
104             object.__setattr__(self, "message_type", MessageType(self.message_type))
105         except (ValueError, TypeError) as exc:
106             raise ProtocolError(ErrorCode.MALFORMED, "Unknown message type") from exc
107         if type(self.generation) is not int or not 0 <= self.generation < 2**63:
108             raise ProtocolError(ErrorCode.MALFORMED, "Invalid execution generation")
109         for label in ("instance_id", "session_id", "request_id"):
110             value = getattr(self, label)
111             if not isinstance(value, str) or len(value) > _MAX_IDENTIFIER:
112                 raise ProtocolError(ErrorCode.MALFORMED, f"Invalid {label}")
113             if value:
114                 validate_segment(value, label)
115         if not isinstance(self.body, dict) or any(not isinstance(key, str) for key in self.body):
116             raise ProtocolError(ErrorCode.MALFORMED, "Message body must be a string-keyed map")
117         if self.message_type in {
118             MessageType.OBSERVATION,
119             MessageType.ACTION,

```

2. ActionProvenance

src/lerobot/inference/contracts.py:152

Excerpt lines 152-168. Line numbers refer to the documented source snapshot.

```

152 class ActionProvenance:
153     """Source identity carried by every queued action, including across merges."""
154
155     capture_time: float
156     task: str
157     task_version: int = 0
158     observation_id: str = ""
159     request_id: str = ""
160     generation: int = 0
161     session_id: str = ""
162     server_instance_id: str = ""
163     artifact_identity: str = ""
164     # The primary IDs identify the incoming prediction. capture_time is the
165     # conservative age bound; this summary retains the oldest original source.
166     oldest_contributor: ActionSource | None = None
167     contributor_count: int = 1
168     contributor_digest: str = ""

```

3. test_evicted_request_identity_cannot_be_replayed

tests/remote_inference/test_session.py:139

Excerpt lines 139-146. Line numbers refer to the documented source snapshot.

```
139 def test_evicted_request_identity_cannot_be_replayed(worker):
140     session = admit(worker)
141     first = action_request(worker, session)
142     assert worker.submit(first).result(2).message_type is MessageType.ACTION
143     for _ in range(128):
144         assert worker.submit(action_request(worker, session)).result(2).message_type is Mess
ageType.ACTION
145     assert first.request_id not in worker._seen
146     assert_error(worker.submit(first).result(2), ErrorCode.STALE)
```

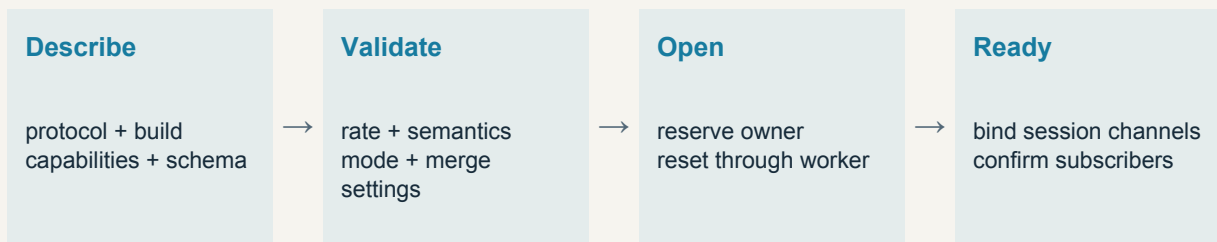
2 · Trace one request

06. Admission establishes a complete contract

2 · TRACE ONE REQUEST

Admission establishes a complete contract

Description, negotiation and readiness all happen before action requests.



Version logs help diagnosis. Protocol and capability checks determine compatibility.

06

RemoteClient.connect obtains a descriptor. admit compares features, action representation, interval and semantics. For alignment/blending it sends required capabilities and exact chunk settings; plain append uses compatible defaults. Alignment and blending require explicit support, even though the protocol major remains 1.

SessionWorker validates the open request and reserves one session. The worker performs reset before acknowledging admission. The client verifies the acknowledgment and establishes session-scoped channels and presence. Data publication waits for subscriber readiness.

Two peers need not have identical Git hashes or package versions. They must satisfy the actual protocol and requested feature contract. The loaded-build log helps reveal an outdated running server; it is not a substitute for negotiation.

Check your understanding

What happens if an older server does not advertise alignment?

Answer

A client requiring alignment fails admission rather than silently falling back to append.

06. Source excerpts - all tabs expanded

1. RemoteClient.admit

src/lerobot/remote_inference/client.py:186

Excerpt lines 186-217 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

186     def admit(
187         self,
188         *,
189         features: tuple[FeatureSpec, ...],
190         action_feature: FeatureSpec,
191         semantics: str,
192         action_interval: float,
193         mode: str,
194     ) -> None:
195         """Validate the robot contract and acquire one exclusive session."""
196         caps = self.capabilities
197         if (
198             features != caps.features
199             or action_feature != caps.action_feature
200             or semantics != self.descriptor["semantics"]
201             or not math.isclose(action_interval, caps.action_interval, rel_tol=1e-6)
202             or ExecutionMode(mode) not in caps.modes
203         ):
204             raise ProtocolError(ErrorCode.INCOMPATIBLE, "Robot features, conventions, cadenc
e or mode differ")
205         required = required_chunk_capabilities(self.chunk_settings)
206         if required:
207             if mode != ExecutionMode.CHUNK:
208                 raise ProtocolError(ErrorCode.INCOMPATIBLE, "Aligned merge requires chunk ex
ecution")
209             advertised = self.descriptor.get("execution_contracts", [])
210             if not isinstance(advertised, list) or any(name not in advertised for name in re
quired):
211                 raise ProtocolError(
212                     ErrorCode.UNSUPPORTED,
213                     "Server does not support the requested chunk alignment/blending contract
; update the server",
214                 )
215             try:
216                 self.blend_indices = validate_chunk_contract(
217                     self.chunk_settings, caps, self.descriptor.get("blendable_components", [

```

2. SessionWorker._validate_open

src/lerobot/remote_inference/server.py:338

Excerpt lines 338-367. Line numbers refer to the documented source snapshot.

```

338     def _validate_open(self, body: dict[str, Any]) -> None:
339         caps = self.runner.capabilities
340         required = body.get("required_capabilities", [])
341         if not isinstance(required, list) or any(
342             not isinstance(name, str) or name not in self.execution_contracts for name in re
quired
343         ):
344             raise ProtocolError(ErrorCode.UNSUPPORTED, "Unknown required protocol capabiliti
es")
345         settings = body.get("chunk_settings", chunk_settings("append", 0, 0.5, []))
346         try:
347             validate_chunk_contract(settings, caps, self.blendable_components)
348         except ValueError as exc:
349             raise ProtocolError(ErrorCode.INCOMPATIBLE, str(exc)) from exc
350         if required != required_chunk_capabilities(settings):
351             raise ProtocolError(ErrorCode.INCOMPATIBLE, "Required capabilities differ from c
hunk_settings")

```

```

352         if settings["chunk_merge"] == "aligned" and body.get("mode") != ExecutionMode.CHUNK:
353             raise ProtocolError(ErrorCode.INCOMPATIBLE, "Aligned merge requires chunk execut
ion")
354         if body.get("expected_artifact") not in (None, self.artifact_identity):
355             raise ProtocolError(ErrorCode.INCOMPATIBLE, "Deployed artifact identity differs"
)
356         if body.get("semantics") != self.semantics:
357             raise ProtocolError(ErrorCode.INCOMPATIBLE, "Robot/action semantic profile diffe
rs")
358         if ExecutionMode(body["mode"]) not in caps.modes:
359             raise ProtocolError(ErrorCode.UNSUPPORTED, "Requested execution mode is unavaila
ble")
360         if not math.isclose(float(body["action_interval"]), caps.action_interval, rel_tol=1e
-6):
361             raise ProtocolError(ErrorCode.INCOMPATIBLE, "Policy action interval differs")
362         features = tuple(FeatureSpec(**value) for value in body["features"])
363         action = FeatureSpec(**body["action_feature"])
364         if features != caps.features or action != caps.action_feature:
365             raise ProtocolError(ErrorCode.INCOMPATIBLE, "Ordered feature schema or semantics
differ")
366         if body.get("encoding") not in {"raw", "jpeg"}:
367             raise ProtocolError(ErrorCode.UNSUPPORTED, "Unsupported image encoding")

```

3. test_unknown_required_capability_rejects_before_session_admission

tests/remote_inference/test_session.py:104

Excerpt lines 104-108. Line numbers refer to the documented source snapshot.

```

104 def test_unknown_required_capability_rejects_before_session_admission(worker):
105     request = open_request(worker)
106     request.body["required_capabilities"] = ["future-observation-history"]
107     assert_error(worker.submit(request).result(2), ErrorCode.UNSUPPORTED)
108     assert worker.session_id is None

```

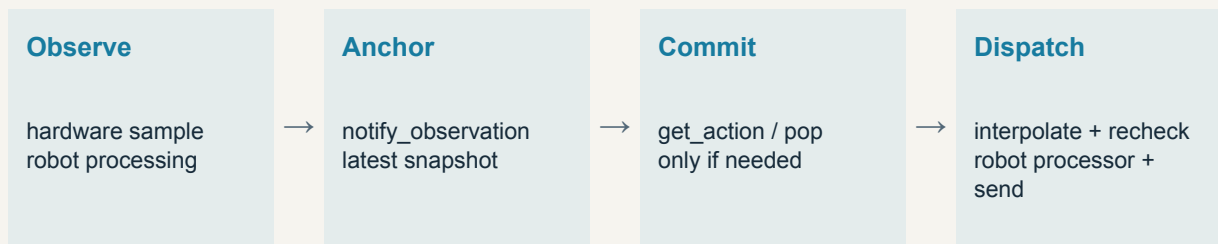
2 · Trace one request

07. A control tick never waits for inference

2 · TRACE ONE REQUEST

A control tick never waits for inference

The strategy offers an observation, then dispatches only permitted local motion.



Observation anchoring precedes the next policy-endpoint commitment on the same thread.

07

The base strategy reads raw hardware on every motor tick; processing and notification are gated to policy-endpoint boundaries. With interpolation enabled, intermediate motor ticks reuse processed observations; they do not manufacture new policy snapshots.

`notify_observation` constructs an owned `ObservationSnapshot` and anchors it to the runtime cursor. `send_next_action` handles the interpolator. It asks for a new canonical endpoint only when needed, checks permission again around dispatch, then processes and sends the local command.

A control tick can continue while `RemoteClient.infer` waits elsewhere. This is not a hard-real-time guarantee: synchronous hardware reads, camera waits and local processing can still delay a tick. Revoking dispatch also clears the interpolator, so an already cached intermediate command cannot keep moving after a hold or fault.

Check your understanding

Is checking permission only when popping a chunk sufficient?

Answer

No. Interpolation can emit more motor commands without another pop, so permission must be checked on every dispatch tick.

07. Source excerpts - all tabs expanded

1. RolloutStrategy._process_observation_and_notify

src/lerobot/rollout/strategies/core.py:171

Excerpt lines 171-194. Line numbers refer to the documented source snapshot.

```

171     def _process_observation_and_notify(
172         self, processors: ProcessorContext, obs_raw: RobotObservation
173     ) -> RobotObservation:
174         """Run the observation processor and notify the engine – throttled to policy ticks.
175
176         Callers are responsible for calling ``robot.get_observation()`` every loop
177         iteration so ``obs_raw`` stays fresh for the action post-processor. This
178         helper gates only the comparatively expensive bits – the processor pipeline
179         and ``engine.notify_observation`` – to fire when the interpolator signals
180         it needs a new action (once per ``interpolation_multiplier`` ticks). On
181         interpolated ticks the cached ``obs_processed`` is reused.
182
183         With ``interpolation_multiplier == 1`` this is equivalent to the unthrottled
184         path: ``needs_new_action()`` is True every tick.
185
186         The cache is implicitly invalidated whenever ``interpolator.reset()`` is
187         called (warmup completion, DAGger phase transitions back to AUTONOMOUS),
188         because reset makes ``needs_new_action()`` return True on the next call.
189         """
190         if self._cached_obs_processed is None or self._require_interpolator().needs_new_acti
on():
191             obs_processed = processors.robot_observation_processor(obs_raw)
192             self._require_engine().notify_observation(obs_processed)
193             self._cached_obs_processed = obs_processed
194         return self._cached_obs_processed

```

2. send_next_action

src/lerobot/rollout/strategies/core.py:443

Excerpt lines 443-474 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

443     def send_next_action(
444         obs_processed: dict,
445         obs_raw: dict,
446         ctx: RolloutContext,
447         interpolator: ActionInterpolator,
448         timer: CycleTimer | None = None,
449     ) -> dict | None:
450         """Dispatch the next action to the robot.
451
452         Pulls the next action tensor from the inference engine, feeds the
453         interpolator, and sends the interpolated action through the
454         ``robot_action_processor`` to the robot. Works identically for
455         sync and async backends – the rollout strategy never needs to branch.
456
457         When *timer* is given, the engine pull and the robot send are timed as the
458         ``infer`` and ``send`` steps of its cadence summary, and a tick with no action
459         to send is counted there. Note that on async backends ``infer`` is only a
460         queue pull – inference runs off-thread, so its latency surfaces as starved
461         ticks rather than as loop-body time.
462
463         Returns the action dict that was sent, or ``None`` if no action was
464         ready (e.g. empty async queue, interpolator not yet primed).
465         """
466         engine = ctx.policy.inference
467         engine.begin_control_tick()
468         features = ctx.data.dataset_features
469         ordered_keys = ctx.data.ordered_action_keys
470         # ``nullcontext`` accepts (and ignores) the section name, so it stands in for
471         # ``timer.section`` verbatim when no timer was passed.

```

```

472     section = timer.section if timer is not None else contextlib.nullcontext
473
474     def dispatch_permitted() -> bool:

```

3. RemoteInferenceEngine.notify_observation

src/lerobot/rollout/inference/remote.py:245

Excerpt lines 245-264. Line numbers refer to the documented source snapshot.

```

245     def notify_observation(self, obs: dict) -> None:
246         sampled = self._robot.observation_time
247         if sampled is None:
248             sampled = time.monotonic()
249         frame = build_dataset_frame(self._features, obs, prefix="observation")
250         mapped = {self._rename_map.get(key, key): value for key, value in frame.items()}
251         if len(mapped) != len(frame):
252             self._fault("Feature mapping contains a collision")
253         return
254         # Mapping is applied once here; server processors require an empty rename map.
255         with self._task_lock:
256             task, version = self._task, self._task_version
257             snapshot = ObservationSnapshot(mapped, sampled, task, version, uuid4().hex)
258             with self._lock:
259                 # Sampling and endpoint commits both belong to this control thread;
260                 # no pop occurs between the hardware read and this anchor. Workers
261                 # may replace the future, but never advance the commitment cursor.
262                 if self.config.chunk_merge == "aligned":
263                     snapshot = self.runtime.anchor_observation(snapshot)
264             self._observation = snapshot

```

2 · Trace one request

08. Concurrency is deliberately bounded

2 · TRACE ONE REQUEST

Concurrency is deliberately bounded

New observations replace the waiting snapshot; they do not form a backlog.

Control thread	latest observation slot → ChunkRuntime / ActionQueue Short shared-state operations; local permission checked each motor tick.
Client worker	ordered control → eligible latest snapshot → infer → accept One inference in flight. Network waits happen here.
Server worker	bounded command handoff → one policy / processor operation Open, reset, action, language and close share serialized ownership.

Bounded queues prevent hidden work accumulation; freshness checks still decide usability.

08

The engine lock protects observation/control/hold coordination, the task lock orders retargets with acceptance, and the runtime lock protects generation, requests and the queue interaction. ActionQueue has its own atomic snapshot/replace operations.

The server has bounded callback handoffs and one outstanding inference admission. Control operations can be queued behind a current model call; they do not reset mutable state concurrently with it.

When reading a method, identify its calling thread and the lock held before interpreting its assignments. A thread-safe container alone does not establish the ordering between observation capture, commitment and result acceptance.

Check your understanding

If inference is slow, should the client save every camera frame for later?

Answer

No. This current-observation runner keeps the latest waiting snapshot and only one in-flight request. A frame backlog would increase source age.

08. Source excerpts - all tabs expanded

1. RemoteInferenceEngine._loop

src/lerobot/rollout/inference/remote.py:374

Excerpt lines 374-405 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

374     def _loop(self) -> None:
375         try:
376             while not self._stop_event.is_set() and not self.failed:
377                 # Control-thread dispatch only queues bounded diagnostics. Log
378                 # from this worker so first-dispatch reporting adds no motor I/O.
379                 with self.runtime.lock:
380                     dispatch_events = list(self.runtime.dispatch_events)
381                     self.runtime.dispatch_events.clear()
382                 for event in dispatch_events:
383                     self._event("first_dispatch", **event)
384                 with self._lock:
385                     control = self._control
386                     observation = self._observation
387                     hold_ready = self._hold_requested and self._hold_acknowledged
388                 if control is not None:
389                     # Language invalidation must be acknowledged by the motor thread
390                     # before submitting any server-side generation operation.
391                     if self._hold_requested and not hold_ready:
392                         self._stop_event.wait(0.002)
393                     continue
394                 operation, generation = control
395                 self.client.control(operation, generation)
396                 with self._lock:
397                     if self._control == control:
398                         self._control = None
399                     continue
400                 if not self.runtime.active or observation is None:
401                     self.runtime.check_deadlines()
402                     self._stop_event.wait(0.002)
403                     continue
404                 if hold_ready:
405                     with self._lock:

```

2. SessionWorker.submit

src/lerobot/remote_inference/server.py:208

Excerpt lines 208-239 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

208     def submit(self, message: Envelope) -> Future[Envelope]:
209         """Validate and enqueue work without running policy code on the caller's thread."""
210         future: Future[Envelope] = Future()
211         with self._lock:
212             try:
213                 if message.message_type is MessageType.DESCRIBE:
214                     future.set_result(
215                         Envelope(
216                             MessageType.DESRIPTOR,
217                             self.instance_id,
218                             request_id=message.request_id,
219                             body=self._descriptor_locked(),
220                         )
221                     )
222                 return future
223                 if message.instance_id != self.instance_id:
224                     raise ProtocolError(ErrorCode.STALE, "Server instance does not match")
225                 if message.message_type is MessageType.OPEN:
226                     if not message.request_id:
227                         raise ProtocolError(ErrorCode.MALFORMED, "Open operation ID is requi
red")
228                 if message.request_id in self._opens:

```

```

229             return self._opens[message.request_id]
230         if self._session is not None:
231             state = self._session_diagnostics_locked()
232             detail = (
233                 f"Deployment owned by session={state['owner']}; "
234                 f"admission_blocker={state['admission_blocker']}"
235                 f"client_present={state['client_present']}"
236                 f"absence_grace_remaining_s={state['absence_grace_remaining_s']}"
237                 f"cleanup_pending={state['cleanup_pending']}"
238                 f"inference_pending={state['inference_pending']}. "
239                 "A new session requires the previous owner's close/reset to fini
sh; "

```

3. ActionQueue.snapshot

src/lerobot/policies/rtc/action_queue.py:127

Excerpt lines 127-139. Line numbers refer to the documented source snapshot.

```

127     def snapshot(self) -> QueueSnapshot:
128         """Copy cursor, both continuations and their provenance under one lock."""
129         with self.lock:
130             return QueueSnapshot(
131                 generation=self._generation,
132                 cursor=self._cursor,
133                 index=self.last_index,
134                 model_actions=None
135                 if self.original_queue is None
136                 else self.original_queue[self.last_index :].clone(),
137                 canonical_actions=None if self.queue is None else self.queue[self.last_index
: ].clone(),
138                 provenance=tuple(self._provenance_queue[self.last_index :]),
139             )

```

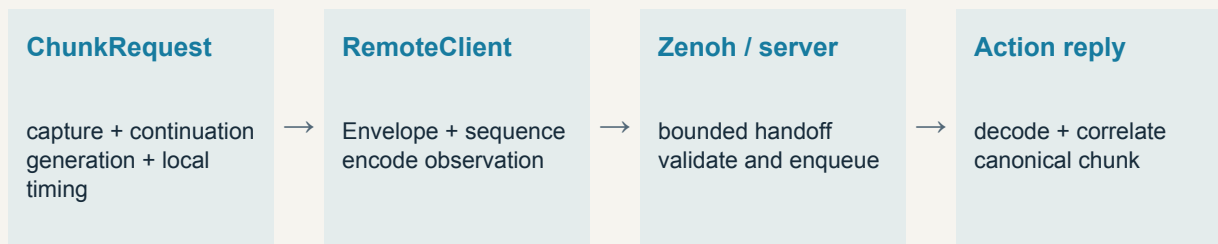
2 · Trace one request

09. The request crosses a bounded transport

2 · TRACE ONE REQUEST

The request crosses a bounded transport

Control uses query/reply; observations and results use session channels.



No unbounded replay queue and no policy work inside transport callbacks.

09

`RemoteClient.infer` constructs an observation request, including the RTC continuation when required. The `MessagePack` codec preserves tensor metadata, enforces configured bounds and uses raw RGB or explicitly selected JPEG encoding. JPEG is lossy; it is a deployment tradeoff, not an exact-byte transport.

The server validates instance/session/generation and data shape, rejects duplicate or non-increasing data requests, and hands accepted work to `SessionWorker`. The envelope header is correlated before the tensor body is decoded; the validated reply becomes an `ActionChunk`.

The transport is not an automatic action retry/replay mechanism. Readiness, bounded waiting and terminal errors are explicit. A client may continue locally buffered valid motion during a transient absence, but a terminal fault cannot be undone by a later packet.

Check your understanding

Why does the server track a monotonic sequence as well as recent request IDs?

Answer

Its request-ID cache is bounded. The sequence rejects a replay even after that ID has aged out of the cache.

09. Source excerpts - all tabs expanded

1. RemoteClient.infer

src/lerobot/remote_inference/client.py:382

Excerpt lines 382-413 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

382     def infer(self, request: ChunkRequest, *, cancelled: Callable[[], bool] = lambda: False)
      -> ActionChunk:
383         """Exchange one action request and validate its identity, execution context and valu
es."""
384         body = self._observation_body(request.observation)
385         model_continuation = request.continuation.model_actions
386         canonical_continuation = request.continuation.canonical_actions
387         body.update(
388             {
389                 "delay": request.delay,
390                 "model_continuation": model_continuation
391                 if model_continuation is not None and len(model_continuation)
392                 else None,
393                 "canonical_continuation": canonical_continuation
394                 if canonical_continuation is not None and len(canonical_continuation)
395                 else None,
396                 "cursor": request.continuation.cursor,
397             }
398         )
399         if self.chunk_settings["chunk_merge"] == "aligned":
400             body["observation_cursor"] = request.observation.action_cursor
401         envelope = Envelope(
402             MessageType.OBSERVATION,
403             self.instance_id,
404             self.session_id,
405             request.generation,
406             request.request_id,
407             body,
408         )
409         response = self._exchange(envelope, "/obs", self._actions, self.config.action_timeou
t_s, cancelled)
410         self._validate_context(response, body, MessageType.ACTION)
411         actions = response.body.get("canonical_actions")
412         model = response.body.get("model_actions")
413         caps = self.capabilities

```

2. SessionWorker.submit

src/lerobot/remote_inference/server.py:208

Excerpt lines 290-321 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

290         _remember(self._controls, message.request_id, future, 32)
291         elif message.message_type in {MessageType.OBSERVATION, MessageType.LANGU
AGE_REQUEST}:
292             if not session.ready or session.closing or session.faulted:
293                 raise ProtocolError(ErrorCode.STALE, "Session is not accepting i
nference")
294             if message.generation != session.generation:
295                 raise ProtocolError(ErrorCode.STALE, "Generation has not been ac
knowledged")
296             if message.request_id in self._seen:
297                 raise ProtocolError(ErrorCode.STALE, "Duplicate or completed req
uest")
298             if session.busy:
299                 raise ProtocolError(
300                     ErrorCode.BUSY, "One inference operation is already outstand
ing"
301                 )
302             self._validate_data(message)
303             if message.body["sequence"] <= session.last_sequence:

```

```

304             raise ProtocolError(ErrorCode.STALE, "Request sequence was already consumed")
305             if self._commands.full():
306                 raise ProtocolError(ErrorCode.BUSY, "Policy command queue is full")
307             session.busy = True
308             session.last_sequence = message.body["sequence"]
309             _remember(self._seen, message.request_id, None)
310         else:
311             raise ProtocolError(ErrorCode.MALFORMED, "Unexpected request message type")
312         self._commands.put_nowait((message, future, time.monotonic()))
313     except (ProtocolError, ValueError, KeyError, TypeError, Full) as exc:
314         code = (
315             exc.code
316             if isinstance(exc, ProtocolError)
317             else ErrorCode.BUSY
318             if isinstance(exc, Full)
319             else ErrorCode.MALFORMED
320         )
321         details = exc.details if isinstance(exc, ProtocolError) else None

```

3. encode_message

src/lerobot/remote_inference/codec.py:279

Excerpt lines 279-295. Line numbers refer to the documented source snapshot.

```

279 def encode_message(envelope: Envelope, limits: CodecLimits = CodecLimits()) -> bytes:
280     """Encode only supported, finite values into an explicitly typed envelope."""
281     require_package("msgpack", "remote")
282     envelope.__post_init__()
283     record = {
284         "version": envelope.version,
285         "message_type": str(envelope.message_type),
286         "instance_id": envelope.instance_id,
287         "session_id": envelope.session_id,
288         "generation": envelope.generation,
289         "request_id": envelope.request_id,
290         "body": envelope.body,
291     }
292     encoded = msgpack.packb(_encode_value(record, _Budget(limits)), use_bin_type=True)
293     if len(encoded) > limits.max_payload_bytes:
294         raise _malformed("Encoded message exceeds byte limit")
295     return encoded

```

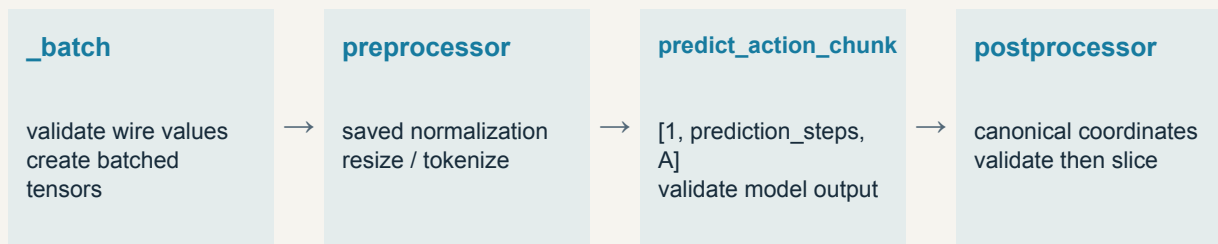
2 · Trace one request

10. PolicyRunner is the model boundary

2 · TRACE ONE REQUEST

PolicyRunner is the model boundary

The direct chunk call must reproduce the policy's inference preparation.



Plain execution uses `execution_steps`. RTC keeps the declared prediction horizon.

10

The runner does not call `select_action`. The direct `predict_action_chunk` implementation must prepare everything it needs without depending on observation history or queues that only `select_action` populates.

It validates the full predicted tensor before postprocessing, saves a detached model-space copy, validates the canonical tensor and then applies the plain execution slice. The current default runner requires the same declared dimensions before and after postprocessing; a padded model width cannot be silently accepted just because a later processor crops it.

Guided RTC requires a local autograd correction. The outer call uses `no_grad` while disabling `inference_mode` for guided RTC, allowing the policy to enable gradients for that correction. This was one of the fixes retained from hardware testing.

Check your understanding

Why does a working synchronous checkpoint not automatically prove remote compatibility?

Answer

`select_action` may collect history, crop a different horizon or perform preparation that direct chunk prediction does not. The remote contract has to describe the actual direct path.

10. Source excerpts - all tabs expanded

1. PolicyRunner.predict

src/lerobot/inference/policy_runner.py:217

Excerpt lines 233-274 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

233         # permits that override; inference_mode would suppress its gradient graph.
234         with torch.inference_mode(mode is not ExecutionMode.RTC_GUIDED), torch.no_grad():
235             prepared = self.preprocessor(self._batch(observation))
236             preprocessed_at = time.perf_counter()
237             kwargs: dict[str, Any] = {}
238             if mode != ExecutionMode.CHUNK:
239                 prefix = self._prepare_continuation(model_continuation, canonical_continuation)
240             on)
241                 if inference_delay < 0:
242                     raise ValueError("RTC delay must be nonnegative.")
243                 if prefix is None and inference_delay:
244                     raise ValueError("RTC delay requires a real continuation prefix.")
245                 if mode == ExecutionMode.RTC_TRAINED:
246                     available = 0 if model_continuation is None else len(model_continuation)
247                     if inference_delay > min(
248                         self.capabilities.training_max_delay, available, self.capabilities.r
249                     tc_horizon
250                     ):
251                         raise ValueError(
252                             "Trained RTC delay exceeds checkpoint or available continuation
253                             limits."
254                         )
255                 kwargs = {"inference_delay": inference_delay, "prev_chunk_left_over": prefix
256             }
257             predicted = self.policy.predict_action_chunk(prepared, **kwargs)
258             predicted_at = time.perf_counter()
259             self._validate_actions(predicted)
260             # A postprocessor may modify its argument. Preserve model coordinates
261             # before invoking the paired canonical postprocessor.
262             original = predicted.detach().clone()
263             canonical = self.postprocessor(predicted)
264             self._validate_actions(canonical)
265             steps = (
266                 self.capabilities.execution_steps
267                 if mode == ExecutionMode.CHUNK
268                 else self.capabilities.prediction_steps
269             )
270             canonical = canonical[0, :steps].detach().to(device="cpu", dtype=torch.float32).
271             clone()
272             model = (
273                 original[0, :steps].detach().to(device="cpu", dtype=torch.float32).clone()
274                 if mode != ExecutionMode.CHUNK
275                 else None
276             )
277             finished = time.perf_counter()
278             return ActionChunk(
279                 model_actions=model,

```

2. PolicyRunner._validate_actions

src/lerobot/inference/policy_runner.py:291

Excerpt lines 291-296. Line numbers refer to the documented source snapshot.

```

291     def _validate_actions(self, actions: torch.Tensor) -> None:
292         expected = (1, self.capabilities.prediction_steps, self.capabilities.action_feature.
293             shape[0])
294         if not isinstance(actions, torch.Tensor) or tuple(actions.shape) != expected:
295             raise ValueError(f"Policy/processor must return action shape {expected}.")
296         if not actions.is_floating_point() or not torch.isfinite(actions).all():
297             raise ValueError("Policy/processor returned non-finite or non-floating actions.")

```

)

3. PreTrainedPolicy.chunk_inference_spec

src/lerobot/policies/pretrained.py:307

Excerpt lines 307-338 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

307     def chunk_inference_spec(self) -> ChunkPolicySpec:
308         """Declare the default current-observation chunk-serving author contract.
309
310         Inheriting this implementation declares that ``predict_action_chunk`` does
311         every preparation needed by direct callers, accepts the complete current
312         observation, and returns ``[B, chunk_size, A]`` in model coordinates. Its
313         canonical processor pair must support whole chunks. ``n_action_steps`` is
314         the ordinary playback slice; RTC retains the full prediction horizon.
315
316         A policy requiring select_action-only preparation, other action layouts,
317         or temporal sampling must override this method and reject unsupported
318         configurations or provide a custom runner. Method presence or a successful
319         warmup is not evidence for this contract. Policy/processor state is allowed
320         only within an exclusive session and reset() must clear it for a new run.
321         """
322         config = self.config
323         if config.n_obs_steps != 1:
324             raise ValueError("Chunk serving requires n_obs_steps=1; observation history is u
325 nsupported.")
326         for field_name in (
327             "observation_delta_indices",
328             "image_observation_delta_indices",
329             "state_observation_delta_indices",
330         ):
331             indices = getattr(config, field_name, None)
332             if indices is not None and list(indices) != [0]:
333                 raise ValueError(f"Chunk serving cannot sample temporal history ({field_name
334 }).")
335         if getattr(config, "temporal_ensemble_coeff", None) is not None:
336             raise ValueError("Chunk serving does not support temporal ensembling.")
337         if getattr(config, "use_visual_memory", False) or getattr(config, "use_proprioceptiv
338 e_memory", False):
339             raise ValueError(
340                 "Chunk serving cannot sample observation memory at the required control cade
341 nce."
342             )

```

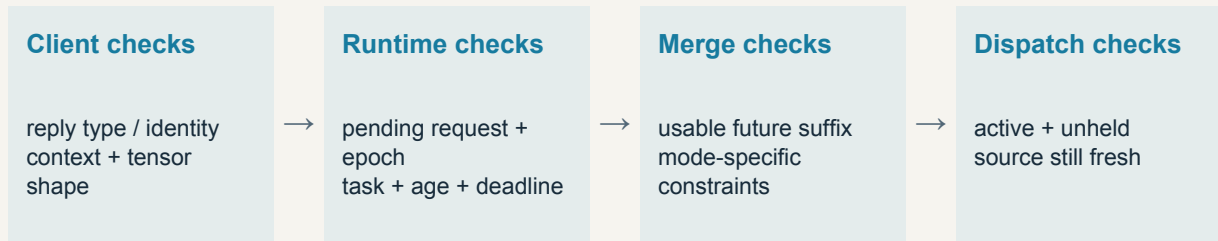
2 · Trace one request

11. Receiving a result does not authorize motion

2 · TRACE ONE REQUEST

Receiving a result does not authorize motion

Correlation, runtime acceptance and dispatch permission are separate gates.



A valid result can still become unusable before its first motor command.

11

RemoteClient checks that the reply belongs to the negotiated context and its request.

ChunkRuntime.accept then checks the currently pending request and execution generation, current task version, elapsed deadlines, source age and chunk structure.

An aligned result with no usable suffix is rejected without destroying an otherwise eligible old future. If no subsequent usable result arrives, the existing age/deadline/exhaustion rules still apply. A stale source-age violation is a terminal fault, which is different from simply rejecting an obsolete result.

At pop and on interpolated ticks, dispatch_allowed checks again. Old buffered actions do not acquire the timestamp of the newest result. Provenance remains attached to the values it actually describes.

Check your understanding

If reset occurs while the network worker is waiting, can that reply refill the new run?

Answer

No. Invalidation changes generation and clears the pending request locally. Acceptance cannot restore the revoked epoch.

11. Source excerpts - all tabs expanded

1. RemoteClient._validate_context

src/lerobot/remote_inference/client.py:449

Excerpt lines 449-460. Line numbers refer to the documented source snapshot.

```

449     def _validate_context(self, response: Envelope, body: dict, expected: MessageType) -> No
ne:
450         context_keys = ["artifact_identity", "observation_id", "capture_time", "task", "task
_version"]
451         if expected is MessageType.ACTION and self.chunk_settings["chunk_merge"] == "aligned
":
452             context_keys.extend(["observation_cursor", "cursor"])
453             if any(type(response.body.get(key)) is not int for key in ("observation_cursor",
"cursor")):
454                 raise ProtocolError(
455                     ErrorCode.MALFORMED, "Result action cursor context differs from its requ
est"
456                 )
457             if response.message_type is not expected or any(
458                 response.body.get(key) != body[key] for key in context_keys
459             ):
460                 raise ProtocolError(ErrorCode.MALFORMED, "Result execution context differs from
its request")

```

2. ChunkRuntime.accept

src/lerobot/inference/execution.py:263

Excerpt lines 263-294 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

263     def accept(self, request: ChunkRequest, chunk: ActionChunk, *, task_version: int) -> boo
l:
264         """Validate context and timing, then atomically merge with original provenance."""
265         with self.lock:
266             self.last_accept = {"accepted": False}
267             self.check_deadlines()
268             if self.failure or self.pending is not request or request.generation != self.gen
eration:
269                 return False
270             self.pending = None
271             if request.observation.task_version != task_version:
272                 return False
273             now = self.clock()
274             elapsed = now - request.submitted_at
275             self._completed += 1
276             if self._completed > 1: # cold start is not a steady-state action latency
277                 self.turnarounds.append(elapsed)
278             if now - request.observation.capture_time > self.max_age:
279                 self.fault("Action result source observation is too old")
280                 return False
281             actions = chunk.canonical_actions
282             model = chunk.model_actions if chunk.model_actions is not None else actions
283             if (
284                 actions.ndim != 2
285                 or model.ndim != 2
286                 or len(actions) != len(model)
287                 or not len(actions)
288                 or type(chunk.execution_steps) is not int
289                 or not 0 < chunk.execution_steps <= len(actions)
290                 or not torch.isfinite(actions).all()
291                 or not torch.isfinite(model).all()
292             ):
293                 self.fault("Invalid action chunk")
294                 return False

```

3. ChunkRuntime.dispatch_allowed

src/lerobot/inference/execution.py:434

Excerpt lines 434-443. Line numbers refer to the documented source snapshot.

```
434     def dispatch_allowed(self) -> bool:
435         """Check permission and the committed endpoint's age on every motor tick."""
436         with self.lock:
437             self.check_deadlines()
438             if self.failure or not self.active or self.held:
439                 return False
440             if self.current is not None and self.clock() - self.current.capture_time > self.
max_age:
441                 self.fault("Dispatched action source observation is too old")
442                 return False
443             return self.current is not None or not self.queue.empty()
```

3 · Read execution

12. The cursor counts committed endpoints

3 · READ EXECUTION

The cursor counts committed endpoints

Commitment happens when pop hands a target to interpolation.

Measured position	Where sensors report the robot is now.
Committed endpoint	A target handed to interpolation; chunk merging cannot replace it.
Queued future	Targets still owned by ActionQueue; eligible for replacement.
Atomic snapshot	Cursor + generation + both action spaces + provenance together.

With interpolation x2, two motor ticks still consume only one policy endpoint.

12

ActionQueue.get_with_provenance advances the commitment cursor. An endpoint is already committed even if the robot is still moving toward it through interpolation. Alignment must not reinterpret that endpoint as replaceable future. A hold, reset or fault can still revoke interpolation and dispatch permission.

The observation is anchored before the next endpoint is popped. A later request also captures an atomic continuation snapshot. These are different moments: aligned trimming uses the observation anchor, including progress that happened before request submission.

Do not use qsize as a proxy for cursor advance: replacement changes queue length. Snapshot consistency also matters for RTC, which needs paired canonical/model continuation and provenance from the same moment.

Check your understanding

Does cursor 103 mean the robot physically reached endpoint 102?

Answer

No. It means 103 endpoints have been committed in that queue epoch. The latest may still be interpolating.

12. Source excerpts - all tabs expanded

1. ActionQueue.get_with_provenance

src/lerobot/policies/rtc/action_queue.py:107

Excerpt lines 107-125. Line numbers refer to the documented source snapshot.

```

107     def get_with_provenance(self) -> tuple[Tensor, str | None, Any] | None:
108         """Commit one action, returning its original request provenance atomically."""
109         with self.lock:
110             if self.queue is None or self.last_index >= len(self.queue):
111                 return None
112
113             if self._task_queue is not None and len(self._task_queue) != len(self.queue):
114                 # A mismatch means some mutation broke the action/task lockstep.
115                 raise RuntimeError(
116                     f"ActionQueue task labels out of sync with actions "
117                     f"({len(self._task_queue)} labels for {len(self.queue)} actions) - "
118                     f"a queue mutation broke the action/task lockstep invariant"
119                 )
120             action = self.queue[self.last_index]
121             task = None if self._task_queue is None else self._task_queue[self.last_index]
122             provenance = self._provenance_queue[self.last_index] if self._provenance_queue else None
123             self.last_index += 1
124             self._cursor += 1
125             return action.clone(), task, provenance

```

2. ActionQueue.snapshot

src/lerobot/policies/rtc/action_queue.py:127

Excerpt lines 127-139. Line numbers refer to the documented source snapshot.

```

127     def snapshot(self) -> QueueSnapshot:
128         """Copy cursor, both continuations and their provenance under one lock."""
129         with self.lock:
130             return QueueSnapshot(
131                 generation=self._generation,
132                 cursor=self._cursor,
133                 index=self.last_index,
134                 model_actions=None
135                 if self.original_queue is None
136                 else self.original_queue[self.last_index :].clone(),
137                 canonical_actions=None if self.queue is None else self.queue[self.last_index :].clone(),
138                 provenance=tuple(self._provenance_queue[self.last_index :]),
139             )

```

3. ChunkRuntime.anchor_observation

src/lerobot/inference/execution.py:143

Excerpt lines 143-155. Line numbers refer to the documented source snapshot.

```

143     def anchor_observation(self, observation: ObservationSnapshot) -> ObservationSnapshot:
144         """Bind a control-thread sample before committing the next endpoint.
145
146         The sampling caller must not pop actions between capture and this call.
147         The rollout control thread owns both operations; a worker may replace
148         the future concurrently but cannot advance this commitment cursor.
149         """
150         with self.lock:
151             return replace(
152                 observation,
153                 action_cursor=self.queue.snapshot().cursor,
154                 execution_generation=self.generation,
155             )

```

3 · Read execution

13. Append and alignment place a chunk differently

3 · READ EXECUTION

Append and alignment place a chunk differently

Illustrative example: capture at cursor 100; accept at cursor 103.

Incoming



Capture anchor = 100

Past slots



3 slots passed before acceptance; cursor = 103

Aligned future



Append instead keeps old future, then queues A0...A7.

Alignment removes 3 committed slots and replaces only the future: incoming A3...A7.

13

Assume an eight-action execution slice A0 through A7 and a capture anchor of 100. One endpoint commits before submission, and two more commit while inference runs. The current cursor is 103, so aligned trim is $103 - 100 = 3$. It is not merely the two endpoints consumed after submission.

Append preserves the remaining old trajectory and places all eight new actions after it. This is bounded to one accepted successor, but the prediction waits and ages. Alignment replaces the uncommitted old future with A3 through A7.

At startup no endpoint has committed, so a slow first inference does not justify skipping the first actions. If trim consumes the entire execution slice, the aligned result is rejected; it never extends to unused prediction steps. The diagram is conceptual, not a measured robot trace.

Check your understanding

If 500 ms pass but the cursor has not advanced, how many steps does plain alignment trim?

Answer

Zero. Plain alignment uses committed progress, not elapsed wall time. Freshness and deadlines are still checked separately.

Expanded interactive example: alignment

Capture cursor stays at 100; the incoming execution slice is A0 through A7. These are illustrative logical slots, not measured hardware positions.

Arrival cursor 100: trim 0; future is A0 through A7. Elapsed time alone does not trim plain alignment.

Arrival cursor 103 (the slide): trim 3; future is A3 through A7. Already committed endpoints are unchanged by this merge.

Arrival cursor 108: no usable incoming suffix. Reject this result while retaining any eligible old future; freshness, deadlines and exhaustion checks still apply.

13. Source excerpts - all tabs expanded

1. ChunkRuntime._accept_aligned

src/lerobot/inference/execution.py:386

Excerpt lines 386-417 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

386     def _accept_aligned(
387         self, request: ChunkRequest, actions: torch.Tensor, provenance: ActionProvenance, no
w: float
388     ) -> bool:
389         snapshot = self.queue.snapshot()
390         anchor = request.observation.action_cursor
391         if anchor is None or request.observation.execution_generation != self.generation:
392             self.fault("Aligned result lacks its observation commitment anchor")
393             return False
394         trim = snapshot.cursor - anchor
395         self.last_accept["trimmed_actions"] = trim
396         if trim < 0 or trim >= len(actions):
397             self.last_accept["rejection"] = "no usable aligned suffix"
398             return False
399         future = actions[trim:].clone()
400         sources = [provenance] * len(future)
401         previous = snapshot.canonical_actions
402         overlap = min(len(previous), len(future)) if previous is not None else 0
403         self.last_accept["overlap_steps"] = overlap
404         if self.blend_indices and max(self.blend_indices) >= future.shape[1]:
405             self.fault("Blend component index exceeds canonical action dimensions")
406             return False
407         if previous is not None and self.blend_weight < 1:
408             for index in range(min(overlap, self.blend_steps)):
409                 old = snapshot.provenance[index]
410                 if (
411                     old is None
412                     or not self._same_context(old, provenance)
413                     or not 0 <= now - old.capture_time <= self.max_age
414                 ):
415                     continue
416                 coordinates = list(self.blend_indices)
417                 future[index, coordinates] = (1 - self.blend_weight) * previous[

```

2. test_observation_anchor_accounts_for_progress_before_and_during_request

tests/inference/test_aligned_execution.py:59

Excerpt lines 59-73. Line numbers refer to the documented source snapshot.

```

59 def test_observation_anchor_accounts_for_progress_before_and_during_request():
60     rt, clock = runtime()
61     initial(rt, clock)
62     rt.pop() # committed endpoint 0, still potentially interpolating toward it
63     clock.now += 0.1
64     source = sample(rt, clock) # action zero assigned to endpoint 1
65     rt.pop() # endpoint 1 committed before request submission
66     request = rt.begin(source)
67     assert request.continuation.cursor == 2
68     rt.pop() # endpoint 2 committed during inference
69     clock.now += 0.001 # wall-time estimate is irrelevant to aligned trimming
70     assert rt.accept(request, result(request, 10), task_version=0)
71     assert rt.last_accept["trimmed_actions"] == 2
72     assert rt.queue.qsize() == 4
73     assert rt.pop()[0].tolist() == [12, 112]

```

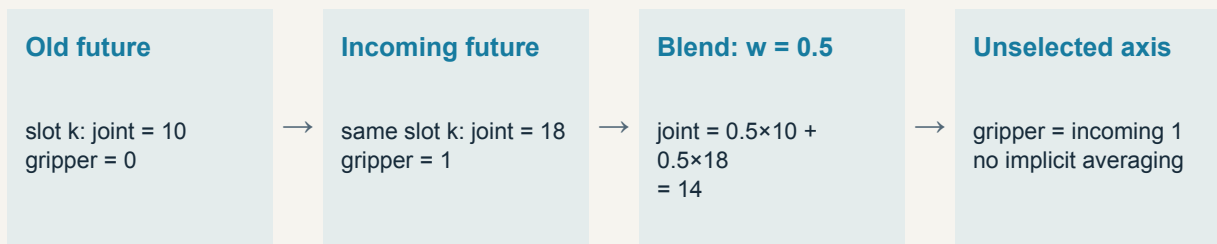
3 · Read execution

14. Blending operates on matching future slots

3 · READ EXECUTION

Blending operates on matching future slots

First align the trajectories. Then average eligible canonical coordinates.



blend_weight is the incoming fraction; **blend_steps** limits eligible overlap.

14

The client explicitly selects `blend_components` from the deployment's allowed continuous canonical coordinates. No gripper is automatically selected. The scalar numbers here are an illustrative arithmetic example, not a recommended joint command.

Only the first $\min(\text{overlap}, \text{blend_steps})$ slots can blend, and only when context matches and the old source remains eligible. Non-selected coordinates remain the incoming values. There is no invented overlap at the end of a short old trajectory. Weight 1 is exact replacement.

Provenance keeps the oldest contributing capture time plus bounded contributor metadata. Repeated blending cannot disguise an old target as newly captured. This is a fixed weighted mixture, not an optimizer or a guarantee of task success. Relative model actions are postprocessed into canonical coordinates before plain blending.

Check your understanding

Can a fresh incoming chunk make an old blended contribution fresh again?

Answer

No. The resulting action carries a conservative age based on the oldest contributing source.

14. Source excerpts - all tabs expanded

1. ChunkRuntime._accept_aligned

src/lerobot/inference/execution.py:386

Excerpt lines 401-432. Line numbers refer to the documented source snapshot.

```

401         previous = snapshot.canonical_actions
402         overlap = min(len(previous), len(future)) if previous is not None else 0
403         self.last_accept["overlap_steps"] = overlap
404         if self.blend_indices and max(self.blend_indices) >= future.shape[1]:
405             self.fault("Blend component index exceeds canonical action dimensions")
406             return False
407         if previous is not None and self.blend_weight < 1:
408             for index in range(min(overlap, self.blend_steps)):
409                 old = snapshot.provenance[index]
410                 if (
411                     old is None
412                     or not self._same_context(old, provenance)
413                     or not 0 <= now - old.capture_time <= self.max_age
414                 ):
415                     continue
416                 coordinates = list(self.blend_indices)
417                 future[index, coordinates] = (1 - self.blend_weight) * previous[
418                     index, coordinates
419                 ] + self.blend_weight * future[index, coordinates]
420                 sources[index] = self._blended_provenance(old, provenance)
421                 self.last_accept["blended_steps"] += 1
422         if self._last_action is not None:
423             self.last_accept["transition_delta"] = (future[0] - self._last_action).tolist()
424         if overlap and previous is not None:
425             self.last_accept["replacement_delta"] = (future[0] - previous[0]).tolist()
426         accepted = self.queue.replace_future(future, sources, snapshot=snapshot)
427         self.last_accept.update(
428             accepted=accepted,
429             usable_actions=len(future),
430             estimated_first_dispatch_source_age_s=now - sources[0].capture_time,
431         )
432         return accepted

```

2. ChunkRuntime._blended_provenance

src/lerobot/inference/execution.py:367

Excerpt lines 367-384. Line numbers refer to the documented source snapshot.

```

367     def _blended_provenance(old: ActionProvenance, new: ActionProvenance) -> ActionProvenanc
368     e:
369         """Summarize all contributing history in constant storage, without refreshing age."""
370         oldest = old.oldest_contributor or ActionSource(old.capture_time, old.observation_id
371         , old.request_id)
372         if new.capture_time < oldest.capture_time:
373             oldest = ActionSource(new.capture_time, new.observation_id, new.request_id)
374         digest = hashlib.sha256(
375             json.dumps(
376                 [old.contributor_digest, old.request_id, old.capture_time, new.request_id, n
377                 ew.capture_time],
378                 separators=(",", ":"),
379             ).encode()
380             ).hexdigest()
381         return replace(
382             new,
383             capture_time=oldest.capture_time,
384             oldest_contributor=oldest,
385             contributor_count=min(old.contributor_count + 1, 2**63 - 1),
386             contributor_digest=digest,
387         )

```

3. test_weighted_blend_matches_future_steps_and_keeps_gripper_incoming

tests/inference/test_aligned_execution.py:283

Excerpt lines 283-301. Line numbers refer to the documented source snapshot.

```

283 def test_weighted_blend_matches_future_steps_and_keeps_gripper_incoming():
284     rt, clock = runtime(blend_steps=2, blend_weight=0.25, blend_indices=(0,))
285     initial(rt, clock)
286     committed = rt.pop()[0].clone()
287     clock.now += 0.1
288     request = rt.begin(sample(rt, clock))
289     assert rt.accept(request, result(request, 10), task_version=0)
290     assert committed.tolist() == [0, 100]
291     values = rt.queue.snapshot().canonical_actions
292     torch.testing.assert_close(values[:3], torch.tensor([[3.25, 110], [4.25, 111], [12, 112]
    ]))
293     assert rt.last_accept["blended_steps"] == 2
294     assert rt.last_accept["overlap_steps"] == 5
295     assert rt.queue.qsize() == 6 # remaining old horizon does not accumulate
296     sources = rt.queue.snapshot().provenance
297     assert sources[0].capture_time == 10
298     assert sources[0].contributor_count == 2
299     assert sources[0].oldest_contributor.request_id
300     assert sources[2].capture_time == 10.1
301     assert sources[2].contributor_count == 1

```

3 · Read execution

15. Refill is a playback threshold

3 · READ EXECUTION

Refill is a playback threshold

The worker asks whether another prediction is needed, not whether a timer fired.

 $\Delta = 1 / \text{action_fps}$

Nominal seconds per policy endpoint

 $L = \text{max recent steady turnaround}$

Up to 100 completed requests; first completion excluded

 $R_{\text{effective}} = \text{max}(\text{refill_seconds}, L + \Delta)$

Measured headroom can raise the configured threshold

request when $\text{queued_steps} \times \Delta \leq R_{\text{effective}}$

Still requires permission, a fresh capture and no in-flight request

Larger refill usually requests earlier. Smaller refill lets more of the trajectory execute.

15

should_request gates both append and aligned chunk mode using remaining queued playback. Append also limits accepted successor storage. Aligned mode does not inherit that successor restriction because it replaces future targets.

The corrected aligned implementation waits for the playback threshold before begin reserves an observation. While it waits, the latest-snapshot slot keeps updating. At request time it can choose a recent capture rather than an observation saved when the worker first became free.

An aligned task change may bypass the playback threshold, but it does not bypass freshness, generation, one-in-flight or dispatch permission. The rolling maximum is a conservative floor based on past measurements, not a promise against the next latency spike.

Check your understanding

Does `refill_seconds=0.2` mean “run inference every 0.2 seconds”?

Answer

No. It means request when roughly 0.2 seconds of queued playback remain, unless the measured floor is larger. Request spacing depends on the chunk horizon and execution progress.

15. Source excerpts - all tabs expanded

1. ChunkRuntime.effective_refill

src/lerobot/inference/execution.py:139

Excerpt lines 139-141. Line numbers refer to the documented source snapshot.

```
139     def effective_refill(self) -> float:
140         """Playback threshold including measured turnaround headroom."""
141         return max(self.refill_seconds, self.turnaround + self.interval)
```

2. ChunkRuntime.should_request

src/lerobot/inference/execution.py:191

Excerpt lines 191-218. Line numbers refer to the documented source snapshot.

```
191     def should_request(self, *, task_version: int | None = None) -> bool:
192         """Check permission and playback need without reserving an observation.
193
194         Aligned task changes may bypass the playback threshold. ``begin`` still
195         validates their fresh capture, generation and commitment anchor.
196         """
197         with self.lock:
198             self.check_deadlines()
199             if self.failure or not self.active or self.held or self.pending is not None:
200                 return False
201             snapshot = self.queue.snapshot()
202             if (
203                 self.chunk_merge == "aligned"
204                 and task_version is not None
205                 and self._last_anchor is not None
206                 and task_version != self._last_anchor[2]
207             ):
208                 return True
209             if self.mode is ExecutionMode.CHUNK and self.chunk_merge == "append":
210                 # One accepted successor beyond the executing chunk, never a chain
211                 # of old-observation predictions hidden behind newer provenance.
212                 sources = {p.request_id for p in snapshot.provenance if p is not None}
213                 if len(sources) > 1:
214                     return False
215                 if self.current and sources and self.current.request_id not in sources:
216                     return False
217             playback = self.queue.qsize() * self.interval
218             return playback <= self.effective_refill + 1e-9
```

3. test_waiting_for_playback_does_not_reserve_an_old_capture

tests/inference/test_aligned_execution.py:109

Excerpt lines 109-131. Line numbers refer to the documented source snapshot.

```
109 def test_waiting_for_playback_does_not_reserve_an_old_capture():
110     rt, clock = runtime(refill_seconds=0.3)
111     first = initial(rt, clock)
112     rt.pop()
113     clock.now += 0.1
114     waiting = sample(rt, clock)
115     assert rt.begin(waiting) is None
116     assert rt.pending is None
117     rt.pop()
118     clock.now += 0.1
119     assert rt.begin(sample(rt, clock)) is None
120     rt.pop()
121     clock.now += 0.1
122     latest = sample(rt, clock)
123     request = rt.begin(latest)
124     assert request is not None
```

```
125     assert request.observation is latest
126     assert request.observation.capture_time > waiting.capture_time > first.observation.captu
re_time
127     assert request.observation.action_cursor == 3
128     rt.pop()
129     assert rt.accept(request, result(request, 10), task_version=0)
130     assert rt.last_accept["trimmed_actions"] == 1
131     assert rt.pop()[0].tolist() == [11, 111]
```

3 · Read execution

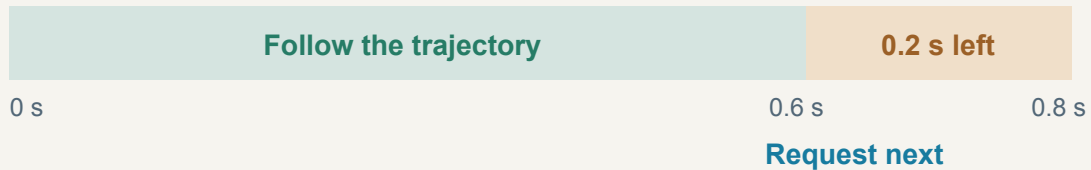
16. Explore request timing

3 · READ EXECUTION

Explore request timing

Idealized aligned playback: 24 actions at 30 Hz, with 100 ms turnaround.

One prediction horizon: $24 / 30 = 0.8$ seconds



Turnaround 0.10 s + one tick 0.033 s < configured refill 0.20 s

Idealized illustration: capture delay, discrete ticks and jitter change actual spacing.

At refill 0.2 s, nominal request spacing is about $0.8 - 0.2 = 0.6$ s.

16

In a regular aligned cycle, the incoming prediction is anchored at capture. Its valid tail ends approximately $H \times \Delta$ later. Requesting when $R_effective$ remains therefore gives approximate request spacing $H \times \Delta - R_effective$, when that value is positive and all other gates allow it.

This approximation assumes capture near submission, regular endpoint commitment, stable turnaround and enough usable suffix. It ignores discrete tick quantization, capture delay, task changes and jitter. The interactive sliders below are explanatory arithmetic, not a runtime simulator or an automatic tuning recommendation.

If the threshold covers the whole execution horizon, playback no longer delays requests; one-in-flight and fresh-advanced-capture gates still apply. Use `request_spacing_s`, `committed_actions_since_request` and `playback_at_submission_s` to see what actually happened on hardware.

Check your understanding

What changes if turnaround rises above the configured refill?

Answer

The measured floor raises the effective threshold. Requests begin earlier, reducing follow-through; a short horizon can still be insufficient.

Expanded interactive example: request timing

For 24 steps at 30 Hz, the execution horizon is 0.800 s and one interval is 0.033 s. Effective refill = max(configured refill, turnaround + one interval). Positive idealized spacing = horizon - effective refill.

Default example: turnaround 0.100 s, configured refill 0.200 s. Effective refill is 0.200 s; idealized request spacing is 0.600 s; arithmetic headroom is 0.100 s.

Measured floor dominates: turnaround 0.250 s, configured refill 0.100 s. Effective refill is 0.283 s; idealized spacing is 0.517 s; arithmetic headroom is 0.033 s.

Threshold covers the horizon: turnaround 0.100 s, configured refill 1.000 s. Playback no longer delays requests. One-in-flight and fresh-capture gates remain; this does not guarantee enough usable actions.

These are the calculator relationships, not measured latency guarantees. Capture delay, discrete ticks, jitter and task changes affect actual request spacing.

16. Source excerpts - all tabs expanded

1. ChunkRuntime.begin

src/lerobot/inference/execution.py:220

Excerpt lines 220-251 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

220     def begin(self, observation: ObservationSnapshot) -> ChunkRequest | None:
221         """Reserve inference against one atomic cursor/continuation snapshot."""
222         with self.lock:
223             if not self.should_request(task_version=observation.task_version):
224                 return None
225             age = self.clock() - observation.capture_time
226             if age < 0 or age > self.max_age:
227                 return None # wait for a fresh capture; startup/dispatch deadlines still ap
ply
228             snapshot = self.queue.snapshot()
229             if self.chunk_merge == "aligned":
230                 if (
231                     observation.execution_generation != self.generation
232                     or observation.action_cursor is None
233                     or observation.action_cursor > snapshot.cursor
234                     or observation.capture_time < self.started_at
235                 ):
236                     return None
237             if self._last_anchor is not None and (
238                 (
239                     observation.task_version == self._last_anchor[2]
240                     and observation.action_cursor <= self._last_anchor[0]
241                 )
242                 or observation.capture_time <= self._last_anchor[1]
243             ):
244                 return None
245             self._last_anchor = (
246                 observation.action_cursor,
247                 observation.capture_time,
248                 observation.task_version,
249             )
250             available = 0 if snapshot.model_actions is None else len(snapshot.model_actions)
251             self.pending = ChunkRequest(

```

2. RemoteInferenceEngine._loop

src/lerobot/rollout/inference/remote.py:374

Excerpt lines 466-497 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

466             configured_refill_s=self.runtime.refill_seconds,
467             effective_refill_s=self.runtime.effective_refill,
468             scheduling="playback_threshold",
469             request_spacing_s=request_spacing,
470             committed_actions_since_request=committed_since_request,
471             task_changed_since_request=task_changed,
472             recent_turnaround_s=self.runtime.turnaround,
473             playback_at_submission_s=request.playback_at_submission,
474             observation_age_at_submission_s=request.submitted_at - observation.captu

```

```

re_time,
475         observation_cursor=observation.action_cursor,
476         submission_cursor=request.continuation.cursor,
477     )
478     self._last_request = (
479         request.generation,
480         request.submitted_at,
481         request.continuation.cursor,
482         observation.task_version,
483     )
484     try:
485         request_generation = request.generation
486
487         def request_cancelled(request_generation: int = request_generation) -> bool:
488             return self._stop_event.is_set() or self.runtime.generation != request_generation
489
490         result = self.client.infer(
491             request,
492             cancelled=request_cancelled,
493         )
494     except RequestCancelled:
495         continue
496     # Acceptance and an operator retarget must have one order: an
497     # old in-flight result cannot pass using a previously read version.

```

3. test_aligned_refill_floor_uses_steady_turnaround_and_excludes_startup

tests/inference/test_aligned_execution.py:134

Excerpt lines 134-153. Line numbers refer to the documented source snapshot.

```

134 def test_aligned_refill_floor_uses_steady_turnaround_and_excludes_startup():
135     rt, clock = runtime(refill_seconds=0.01)
136     first = rt.begin(sample(rt, clock))
137     clock.now += 0.8
138     assert rt.accept(first, result(first), task_version=0)
139     assert rt.effective_refill == pytest.approx(0.1)
140     for _ in range(5):
141         rt.pop()
142     request = rt.begin(sample(rt, clock))
143     assert request is not None
144     clock.now += 0.25
145     assert rt.accept(request, result(request, 10), task_version=0)
146     assert rt.turnaround == pytest.approx(0.25)
147     assert rt.effective_refill == pytest.approx(0.35)
148     for _ in range(2):
149         rt.pop()
150     assert not rt.should_request() # 0.4 seconds remain
151     rt.pop()
152     assert rt.should_request() # 0.3 seconds remain: the measured floor dominates
153     assert rt.begin(sample(rt, clock)) is not None

```

3 · Read execution

17. RTC changes the prediction itself

3 · READ EXECUTION

RTC changes the prediction itself

Plain alignment edits execution; RTC conditions inference on continuation.

Plain aligned

current observation → normal prediction → trim / replace / optional blend

No RTC model support is needed; canonical future targets are merged locally.

RTC

observation + continuation + delay → conditioned prediction → guarded replacement

The model must implement the RTC interface and the deployment must enable that mode.

Canonical continuation preserves physical meaning; model continuation preserves model coordinates.

17

RTC carries paired continuation spaces. For relative-action preprocessing, PolicyRunner reanchors canonical continuation against the current state and re-normalizes it instead of reusing offsets from an old observation.

Guided RTC and trained RTC are distinct capabilities. Trained RTC also has a checkpoint delay bound; the runtime rejects progress beyond what the request could condition on. RTC trimming uses its submission snapshot and timing/progress rules, not plain alignment's capture-anchor rule.

In the current shared runtime, RTC trim is $\min(\text{ceil}(\text{elapsed}/\Delta), \text{cursor advance})$. Trained-overlap checks use the larger measured/progress bound. Plain blending is not automatically layered onto RTC. SmolVLA RTC was physically exercised; other declarations are not blanket checkpoint validation.

Check your understanding

Would adding `blend_steps` give an unsupported model RTC capability?

Answer

No. Blending is a local plain-chunk merge operation. RTC requires model-side continuation conditioning.

17. Source excerpts - all tabs expanded

1. PolicyRunner._prepare_continuation

src/lerobot/inference/policy_runner.py:298

Excerpt lines 298-320. Line numbers refer to the documented source snapshot.

```

298     def _prepare_continuation(
299         self, model: torch.Tensor | None, canonical: torch.Tensor | None
300     ) -> torch.Tensor | None:
301         if model is None or model.numel() == 0:
302             return None
303         width = self.capabilities.action_feature.shape[0]
304         if model.ndim != 2 or model.shape[1] != width or not torch.isfinite(model).all():
305             raise ValueError("Invalid model-space continuation.")
306         device = self.policy.config.device or "cpu"
307         if self._relative_step is not None:
308             if canonical is None or canonical.shape != model.shape or not torch.isfinite(canonical).all():
309                 raise ValueError("Relative RTC requires matching finite canonical continuation.")
310             state = self._relative_step.get_cached_state()
311             if state is None:
312                 raise ValueError("Relative RTC preprocessor did not cache the current raw state.")
313             model = reanchor_relative_rtc_prefix(
314                 canonical, state, self._relative_step, self._normalizer_step, device
315             )
316             horizon = self.capabilities.rtc_horizon
317             model = model.to(device)
318             if len(model) < horizon:
319                 model = torch.cat([model, model[-1:].expand(horizon - len(model), -1)])
320             return model[:horizon]

```

2. ChunkRuntime.accept

src/lerobot/inference/execution.py:263

Excerpt lines 303-334 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

303         has_previous = available is not None and len(available) > 0
304         if self.mode is ExecutionMode.RTC_TRAINED and not trained_overlap_valid(
305             request.delay, max(measured, progress), self.training_max_delay, has_previous
306         ):
307             return False
308         trim = min(measured, progress) if self.mode is not ExecutionMode.CHUNK else 0
309         if trim >= len(actions):
310             self.fault("Inference completed beyond its usable continuation horizon")
311             return False
312         provenance = replace(
313             chunk.provenance,
314             request_id=request.request_id,
315             generation=request.generation,
316             capture_time=request.observation.capture_time,
317             task=request.observation.task,
318             task_version=request.observation.task_version,
319             observation_id=request.observation.observation_id,
320         )
321         self.last_accept.update(
322             turnaround_s=elapsed,
323             playback_at_submission_s=request.playback_at_submission,
324             timing_margin_s=request.playback_at_submission - elapsed if self.has_execute
325         )
326         cursor_advance=progress,
327         trimmed_actions=trim,
328         overlap_steps=0,
329         blended_steps=0,

```

```
329         source_age_at_acceptance_s=now - provenance.capture_time,  
330         remaining_old_playback_s=self.queue.qsize() * self.interval,  
331         estimated_first_dispatch_source_age_s=now  
332         - provenance.capture_time  
333         + (  
334             self.queue.qsize() * self.interval
```

3. trained_overlap_valid

src/lerobot/inference/execution.py:57

Excerpt lines 57-59. Line numbers refer to the documented source snapshot.

```
57 def trained_overlap_valid(conditioned: int, measured: int, maximum: int, has_previous: bool)  
   -> bool:  
58     """Require elapsed overlap to fit both the conditioning and training range."""  
59     return not has_previous or measured <= min(conditioned, maximum)
```

4 · Review lifecycle

18. Runtime deadlines and presence grace

4 · REVIEW LIFECYCLE

Runtime deadlines and presence grace

None of these replaces sequence or generation checks.

Startup timeout	Bound the wait for the first usable action after starting/resuming.
Action request deadline	Bound a pending inference even if the worker is blocked waiting.
Maximum source age	Reject old captures and stop dispatch of actions that have become stale.
Presence grace Default 10 s	Release abandoned server ownership after serialized cleanup.

Fresh local timestamps do not prove that a camera delivered a new sensor frame.

18

Action/startup deadlines and source age are client-local monotonic checks. The server also times its own operations. Server durations are useful diagnostics, but cannot be subtracted directly from a client clock reading.

ObservationSnapshot.capture_time records local acquisition provenance. The present implementation does not propagate comprehensive per-camera sensor timestamps or detect every repeated-frame stall. A new wrapper read may still contain cached camera data. That limitation remains deferred.

The server's idle_timeout_s is a presence/initial-handshake grace, currently defaulting to 10 seconds. It is not a motion recovery window: local action exhaustion or age limits can terminate the client much earlier.

Check your understanding

If the session is still inside its presence grace, may the client execute an expired action?

Answer

No. Ownership grace and local dispatch eligibility are independent.

18. Source excerpts - all tabs expanded

1. ChunkRuntime.check_deadlines

src/lerobot/inference/execution.py:180

Excerpt lines 180-189. Line numbers refer to the documented source snapshot.

```

180     def check_deadlines(self) -> None:
181         """Enforce local action/startup bounds even while network work is blocked."""
182         with self.lock:
183             if self.failure or not self.active or self.held:
184                 return
185             now = self.clock()
186             if self.pending is not None and now - self.pending.submitted_at > self.action_timeout:
187                 self.fault("Action request deadline exceeded")
188             elif not self.has_executed and now - self.started_at > self.startup_timeout:
189                 self.fault("Initial action deadline exceeded")

```

2. ThreadSafeRobot.get_observation

src/lerobot/rollout/robot_wrapper.py:49

Excerpt lines 49-63. Line numbers refer to the documented source snapshot.

```

49     def get_observation(self) -> dict[str, Any]:
50         with self._lock:
51             sampled_at = time.monotonic()
52             try:
53                 observation = self._robot.get_observation()
54                 self._observed_positions = {
55                     key: float(observation[key])
56                     for key in self.action_features
57                     if key.endswith(".pos") and key in observation
58                 }
59             except Exception as exc:
60                 self._record_hardware_failure("get_observation", exc)
61                 raise
62             self._observation_time = sampled_at
63             return observation

```

3. ExecutionConfig

src/lerobot/remote_inference/configs.py:26

Excerpt lines 26-36. Line numbers refer to the documented source snapshot.

```

26 class ExecutionConfig:
27     """Deployment-owned modes, timing bounds and warmup settings."""
28
29     supported_modes: list[str] = field(default_factory=lambda: ["chunk"])
30     action_fps: float = 30.0
31     rtc: RTConfig = field(default_factory=RTConfig)
32     action_deadline_s: float = 5.0
33     idle_timeout_s: float = 10.0
34     warmup_calls: int = 2
35     # Explicit continuous canonical coordinates; never infer gripper suitability.
36     blendable_components: list[str] = field(default_factory=list)

```

4 · Review lifecycle

19. Retargeting and reset are different operations

4 · REVIEW LIFECYCLE

Retargeting and reset are different operations

A new instruction does not automatically mean a new session.

Retarget

set_task → task_version advances → old in-flight reply becomes ineligible

Committed motion is not rewritten. Aligned mode can request the new task before the normal refill threshold.

Reset / planned invalidation

revoke local motion → generation advances → serialized server control ACK

Fresh-generation action work waits for acknowledgment; a full reset also clears model/processor state.

Invalidating motion is immediate locally; interrupting a GPU forward call is not assumed.

19

set_task updates the task and version. It does not flush every queued/committed target; they retain their original provenance. This lets execution continue until an eligible replacement arrives. The task lock gives retarget and result acceptance one order.

Runtime.invalidate increments generation, clears queued motion and the pending request, and resets local execution anchors. Remote engine controls serialize reset/invalidate requests to the server. New-generation work waits until the control acknowledgment.

PolicyRunner.reset(full=False) drops queued policy actions while preserving planner state. A full reset also resets policy and processor pairs. This distinction supports language/planner continuity without permitting old motion to leak into the resumed run.

Check your understanding

Can reset safely run on a network callback while the model is still executing?

Answer

No. Local permission is revoked immediately, but the model reset must wait on the exclusive server worker.

19. Source excerpts - all tabs expanded

1. RemoteInferenceEngine.set_task

src/lerobot/rollout/inference/remote.py:210

Excerpt lines 210-216. Line numbers refer to the documented source snapshot.

```
210     def set_task(self, task: str) -> bool:
211         if len(task) > self.client.descriptor["limits"]["max_input_chars"]:
212             raise ValueError("Instruction exceeds deployed text limit")
213         changed = super().set_task(task)
214         if changed:
215             self._event("task", task=task, task_version=self.task_version)
216         return changed
```

2. RemoteInferenceEngine._invalidate

src/lerobot/rollout/inference/remote.py:182

Excerpt lines 182-192. Line numbers refer to the documented source snapshot.

```
182     def _invalidate(self, operation: str) -> None:
183         with self._lock:
184             generation = self.runtime.invalidate()
185             self._observation = None
186             self._hold_requested = False
187             self._hold_acknowledged = False
188             # A later motion invalidation must not erase a pending full reset.
189             if self._control is not None and self._control[0] == "reset":
190                 operation = "reset"
191             self._control = (operation, generation)
192             self._event(operation)
```

3. test_reset_acknowledges_only_after_current_model_call_and_reset

tests/remote_inference/test_session.py:175

Excerpt lines 175-197. Line numbers refer to the documented source snapshot.

```
175 def test_reset_acknowledges_only_after_current_model_call_and_reset(worker):
176     session = admit(worker)
177     entered, release = block_predict(worker)
178     try:
179         action = worker.submit(action_request(worker, session))
180         assert entered.wait(2)
181         reset_request = control_request(worker, session, 1, "reset")
182         reset = worker.submit(reset_request)
183         assert worker.submit(reset_request) is reset
184         assert not reset.done()
185         assert worker.runner.policy.resets == 2
186         assert_error(worker.submit(action_request(worker, session, generation=1)).result(2),
187                      ErrorCode.STALE)
187         release.set()
188         assert action.result(2).message_type is MessageType.ACTION
189         assert reset.result(2).body["applied_generation"] == 1
190         assert worker.runner.policy.resets == 3
191         assert_error(worker.submit(action_request(worker, session)).result(2), ErrorCode.STALE)
192     except AssertionError:
193         assert (
194             worker.submit(action_request(worker, session, generation=1)).result(2).message_type
195             is MessageType.ACTION
196         )
197     finally:
198         release.set()
```

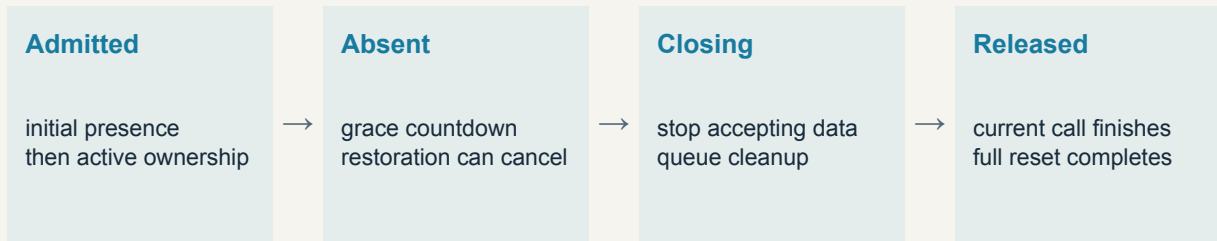
4 · Review lifecycle

20. Session cleanup preserves model ownership

4 · REVIEW LIFECYCLE

Session cleanup preserves model ownership

A disappeared process cannot be replaced while its model call is still mutating state.



Grace expiry schedules cleanup; it does not forcibly free a busy model.

20

SessionWorker distinguishes waiting for initial presence from a later disconnect. This removed the misleading admitted/absent/restored log pattern at every normal connection.

Server-side cleanup can be cancelled if client presence returns within grace before closing begins. This does not restore client inference permission: RemoteClient.present latches any observed server-token loss, so returning server presence cannot enable new requests. Valid buffered motion may drain until local limits stop it. A fresh client process is a new owner and does not adopt the old session.

Clean close releases ownership after worker cleanup. Abrupt disappearance waits the grace and then follows the same serialized cleanup path. A hung model call still owns its state after grace; admitting another client at that point would violate isolation. There is no automatic motion recovery after a terminal fault.

Check your understanding

Should a new client succeed exactly 10 seconds after killing the old one?

Answer

Not necessarily. Presence detection, queueing and the current model call/reset must finish. The grace is not a guaranteed re-admission deadline.

20. Source excerpts - all tabs expanded

1. SessionWorker.expire

src/lerobot/remote_inference/server.py:604

Excerpt lines 604-635 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

604     def expire(self, *, present: bool) -> None:
605         """Bound initial presence and later absence; release only through the worker."""
606         with self._lock:
607             session = self._session
608             if session is None or session.closing:
609                 return
610             now = time.monotonic()
611             if present:
612                 if not session.presence_established:
613                     logger.info(
614                         "Session initial presence established instance=%s session=%s",
615                         self.instance_id,
616                         session.identity,
617                     )
618                 elif session.present is False:
619                     logger.info(
620                         "Session presence restored instance=%s session=%s; absence cleanup c
ancelled",
621                         self.instance_id,
622                         session.identity,
623                     )
624                 session.present = True
625                 session.presence_established = True
626                 session.absent_since = None
627                 session.cleanup_queue_blocked = False
628                 return
629             session.present = False
630             if session.absent_since is None:
631                 session.absent_since = now
632                 logger.info(
633                     (
634                         "Session client absent instance=%s session=%s grace_s=%.3f inference
_pending=%s"
635                         if session.presence_established

```

2. SessionWorker._execute

src/lerobot/remote_inference/server.py:432

Excerpt lines 432-463 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

432     def _execute(self, message: Envelope) -> Envelope:
433         session = self._session
434         if session is None:
435             return message.error(ErrorCode.STALE, "Session closed")
436         if message.message_type is not MessageType.OPEN and message.session_id != session.id
entity:
437             return message.error(ErrorCode.STALE, "Session ownership changed before executio
n")
438         if message.message_type is MessageType.OPEN:
439             self.runner.reset(full=True)
440             session.ready = True
441             logger.info("Session admitted instance=%s session=%s", self.instance_id, session
.identity)
442         return Envelope(
443             MessageType.ACCEPTED,
444             self.instance_id,
445             session.identity,
446             0,
447             message.request_id,
448             {**self.descriptor, "mode": session.mode.value, "chunk_settings": session.ch

```

```

        unk_settings},
449    )
450    if message.message_type is MessageType.CONTROL:
451        operation = message.body["operation"]
452        if operation != "close" and message.generation < session.generation:
453            return message.error(ErrorCode.STALE, "Control generation became obsolete wh
ile queued")
454        if operation == "status" and message.generation != session.generation:
455            return message.error(ErrorCode.STALE, "Status generation does not match")
456        if operation == "close" or (
457            operation in {"reset", "invalidate"} and message.generation > session.genera
tion
458        ):
459            self.runner.reset(full=operation != "invalidate")
460            session.generation = message.generation
461            response = message.reply(
462                MessageType.ACK, {"operation": operation, "applied_generation": session.gene
ration}
463            )

```

3. test_absence_cleanup_never_reuses_a_blocked_model_after_grace

tests/remote_inference/test_session.py:355

Excerpt lines 355-382 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

355 def test_absence_cleanup_never_reuses_a_blocked_model_after_grace(worker, cleanup_clock):
356     session = admit(worker)
357     worker.expire(present=True)
358     entered, release = block_predict(worker)
359     try:
360         action = worker.submit(action_request(worker, session))
361         assert entered.wait(2)
362         worker.expire(present=False)
363         cleanup_clock[0] += 10
364         worker.expire(present=False)
365         state = worker.descriptor["session"]
366         assert state["owner"] == session
367         assert state["absence_grace_remaining_s"] == 0.0
368         assert state["cleanup_reason"] == "client_absence"
369         assert state["admission_blocker"] == "unfinished_inference"
370         # Neither additional absence nor a late presence token revokes an ordered close.
371         cleanup_clock[0] += 300
372         worker.expire(present=False)
373         worker.expire(present=True)
374         assert_error(worker.submit(open_request(worker)).result(2), ErrorCode.BUSY)
375         assert worker.runner.policy.resets == 2
376         after_cleanup = worker.submit(control_request(worker, session, 0, "status"))
377         assert not after_cleanup.done()
378         release.set()
379         assert_error(action.result(2), ErrorCode.TIMEOUT)
380         assert_error(after_cleanup.result(2), ErrorCode.STALE)
381         assert admit(worker) != session
382     finally:

```

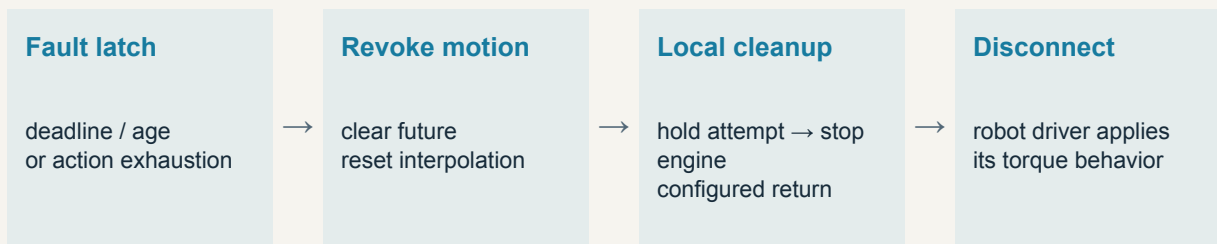
4 · Review lifecycle

21. Terminal fault becomes local shutdown

4 · REVIEW LIFECYCLE

Terminal fault becomes local shutdown

The robot can return to its initial pose even when the server is gone.



Return requires enabled configuration, a captured initial pose and no known hardware I/O failure.

21

ChunkRuntime.fault invalidates motion and latches failure. Later reset/invalidate cannot clear that terminal latch. The engine reports the fault. It lets the control thread issue a local hold before acknowledge_hold signals global shutdown; then the controller ends the segment and runs teardown.

Teardown clears interpolation, attempts a local hold when appropriate, stops the engine, then honors return_to_initial_position when its prerequisites hold. A known robot read/send/hold failure suppresses further shutdown movement. Returning uses local hardware and does not need the server.

Disconnect may remove torque according to the driver configuration. A transient hold command is not a promise of indefinite pose retention. The focused hardware test passed after the revised return behavior; broader hardware teardown hardening remains a separate concern.

Check your understanding

Does restarting the server restore motion after Active motion buffer exhausted?

Answer

No. The client has latched a terminal fault and is shutting down. A later run needs a new client/session.

21. Source excerpts - all tabs expanded

1. ChunkRuntime.fault

src/lerobot/inference/execution.py:172

Excerpt lines 172-178. Line numbers refer to the documented source snapshot.

```
172     def fault(self, reason: str) -> None:
173         """Latch a terminal fault; invalidate does not clear this latch."""
174         with self.lock:
175             if self.failure is None:
176                 self.failure = reason
177                 self.invalidate(held=True)
178                 self.active = False
```

2. RolloutStrategy._teardown_hardware

src/lerobot/rollout/strategies/core.py:227

Excerpt lines 227-258 (method continues in the original file). Line numbers refer to the documented source snapshot.

```
227     def _teardown_hardware(self, hw: HardwareContext, return_to_initial_position: bool = True) -> None:
228         """End policy execution, perform configured local shutdown movement, and disconnect.
229
230         A terminal inference fault does not imply failed local hardware. Honor
231         the return setting in that case; never home through a known I/O failure.
232         Disconnect torque behavior belongs to the robot driver, not the hold.
233         """
234         wrapper = hw.robot_wrapper
235         if self._interpolator is not None:
236             self._interpolator.reset()
237         self._cached_obs_processed = None
238         try:
239             if self._engine is not None:
240                 logger.info("Stopping inference engine...")
241                 if self._engine.failed and wrapper.supports_hold and wrapper.hardware_failure
242                     is None:
243                     try:
244                         wrapper.hold()
245                     except Exception:
246                         logger.exception("Fault hold failed; skipping further shutdown movement")
247                 self._engine.stop()
248             finally:
249                 robot = wrapper.inner
250                 try:
251                     if robot.is_connected:
252                         try:
253                             if wrapper.hardware_failure is not None:
254                                 logger.warning(
255                                     "Skipping return-to-initial-position: robot I/O failed: %s",
256                                     wrapper.hardware_failure,
257                                 )
258                             elif not return_to_initial_position:
259                                 logger.info("Skipping return-to-initial-position: disabled by config")
```

3. test_terminal_fault_shutdown_orders_hold_stop_home_disconnect

tests/test_remote_rollout.py:298

Excerpt lines 298-318. Line numbers refer to the documented source snapshot.

```

298 def test_terminal_fault_shutdown_orders_hold_stop_home_disconnect(monkeypatch):
299     engine = GateEngine()
300     engine.failed = True
301     ctx, robot = make_dispatch_context(engine)
302     strategy = BaseStrategy(BaseStrategyConfig())
303     strategy._engine = engine
304     strategy._interpolator = ActionInterpolator()
305     strategy._interpolator.add(torch.tensor([123.0]))
306     strategy._cached_obs_processed = {"joint.pos": 123.0}
307     events = []
308     monkeypatch.setattr(robot, "send_action", lambda action: events.append(("action", action
    .copy()))
309     monkeypatch.setattr(engine, "stop", lambda: events.append(("engine_stop", None)))
310     monkeypatch.setattr(robot, "disconnect", lambda: events.append(("disconnect", None)))
311     monkeypatch.setattr("lerobot.rollout.strategies.core.precise_sleep", lambda _: None)
312     strategy._teardown_hardware(ctx.hardware)
313     assert events[0] == ("action", {"joint.pos": 7.0})
314     assert events[1] == ("engine_stop", None)
315     assert events[-2] == ("action", {"joint.pos": 0.0})
316     assert events[-1] == ("disconnect", None)
317     assert strategy._interpolator.get() is None
318     assert strategy._cached_obs_processed is None

```

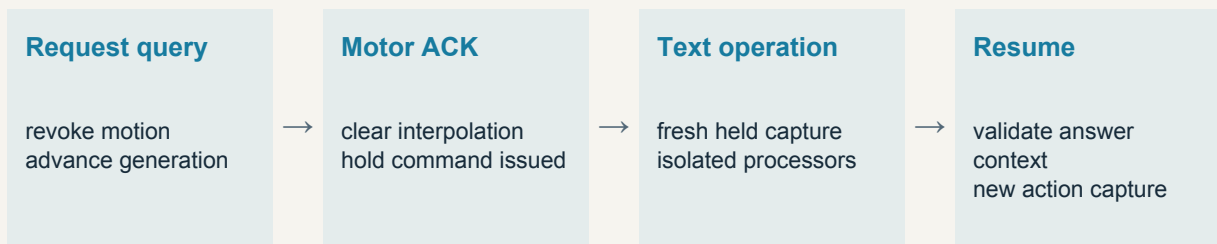
4 · Review lifecycle

22. Language is a planned hold path

4 · REVIEW LIFECYCLE

Language is a planned hold path

Text work shares the policy worker, so motion is explicitly suspended.



Implemented and software-tested; real text-capable checkpoint acceptance is still deferred.

22

The acknowledgment means the local hold command was issued, not that sensors proved physical stillness. The control thread must acknowledge the hold before the worker sends the generation control and language request. The query requires a new capture after the control thread acknowledges issuing the hold. The worker receives the server generation-control acknowledgment before submitting language work. A queued full reset must not be downgraded by a later hold request.

PolicyRunner uses an isolated language processor pair, while the policy is still accessed through the same serialized worker. Query IDs, task versions and autosteering intent prevent obsolete answers from changing the current task.

After language work completes without a terminal fault, the old observation is discarded so action inference receives a post-query capture. A recoverable text/execution error can return an error answer and resume; timeout or uncertain protocol state faults the rollout. Autosteering requires a real dispatched action after applying a subtask before another subtask query. Natural-language task conditioning alone does not mean a policy implements VQA.

Check your understanding

Why not let the robot finish a long chunk while a slow VQA call runs?

Answer

The agreed behavior is a planned local hold, with bounded serialized language work and fresh resumption. Concurrent action/text scheduling is outside this implementation.

22. Source excerpts - all tabs expanded

1. RemoteInferenceEngine._request_language_hold

src/lerobot/rollout/inference/remote.py:282

Excerpt lines 282-293. Line numbers refer to the documented source snapshot.

```

282     def _request_language_hold(self) -> None:
283         with self._lock:
284             if not self.runtime.active or self._hold_requested or self.failed:
285                 return
286             generation = self.runtime.invalidate(held=True)
287             self._hold_requested = True
288             self._hold_acknowledged = False
289             self._hold_started = time.monotonic()
290             self._observation = None
291             operation = "reset" if self._control is not None and self._control[0] == "reset"
292         else "invalidate"
293             self._control = (operation, generation)
294             self._event("planned_hold")

```

2. RemoteInferenceEngine._loop

src/lerobot/rollout/inference/remote.py:374

Excerpt lines 403-434 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

403         continue
404     if hold_ready:
405         with self._lock:
406             # Reset/pause may arrive after the initial loop snapshot.
407             # A new-generation query must wait for its ordered control
408             # acknowledgement, just like an action request.
409             eligible = (
410                 self._control is None
411                 and self.runtime.active
412                 and self._hold_requested
413                 and self._hold_acknowledged
414                 and observation is self._observation
415                 and observation.capture_time >= self._hold_started
416             )
417             generation = self.runtime.generation
418             if eligible:
419                 self._query_observation, self._query_generation = observation, g
420         generation
421         if not eligible:
422             self._stop_event.wait(0.002)
423             continue
424         self._service_query({})
425         with self._lock:
426             if self.runtime.generation == generation and not self.failed:
427                 self.runtime.held = False
428                 self.runtime.started_at = time.monotonic()
429                 self._hold_requested = False
430                 self._hold_acknowledged = False
431                 self._observation = None # action always gets a post-query capt
432         ure
433         continue
434     if self.has_pending_query or not self.client.present:
435         self.runtime.check_deadlines()
436         self._stop_event.wait(0.002)

```

3. test_query_waits_for_hold_acknowledgment_and_new_capture

tests/remote_inference/test_engine.py:159

Excerpt lines 159-183. Line numbers refer to the documented source snapshot.

```
159 def test_query_waits_for_hold_acknowledgment_and_new_capture(session):
160     engine, client = session
161     engine.resume()
162     capture(engine)
163     assert engine.ask("What is visible?")
164     engine.start()
165     assert not client.control_started.wait(0.02)
166     assert not client.text_started.is_set()
167     assert not engine.dispatch_allowed()
168     engine.acknowledge_hold()
169     assert client.control_started.wait(2)
170     assert not client.text_started.wait(0.02), "the pre-hold capture cannot condition text"
171     capture(engine)
172     assert client.text_started.wait(2)
173     client.text_release.set()
174     assert wait_for(lambda: bool(engine._ready_answers))
175     delivered = []
176     engine.set_answer_observer(delivered.append)
177     assert delivered == []
178     engine.pump_query()
179     assert delivered[0].answer == "an answer"
180     assert not client.action_started.is_set(), "action resumption requires another fresh cap
ture"
181     capture(engine)
182     assert client.action_started.wait(2)
183     assert client.max_active_calls == 1
```

5 · Prepare your review

23. Policy compatibility is an execution contract

5 · PREPARE YOUR REVIEW

Policy compatibility is an execution contract

A small declaration is appropriate only when it describes the policy truthfully.

ACT	Default direct-chunk contract; temporal ensembling must be disabled.
SmoIVLA / XVLA	Targeted input validation preserves existing resize, masks and state preparation.
LaWAM	Declares its returned action horizon and current-only inference preparation.
History-dependent policies	Need real observation / executed-action semantics; bypassing guards is insufficient.

The policy audit separates source-compatible candidates from physically exercised checkpoints.

23

`chunk_inference_spec` declares prediction/execution lengths and available modes.

`validate_chunk_input_features` distinguishes accepted canonical inputs from the checkpoint's existing preparation. The exact negotiated wire schema remains strict.

The inherited contract rejects history and certain stateful preparation patterns it cannot represent. LingBot, for example, consumes frame/executed-action history and model cache state; constant-width current snapshots are not enough. Some other policies only need a targeted horizon declaration.

Read `policy_support_audit.md` for all 21 built-in families and their caveats. The runner currently uses direct construction, not a generic YAML adapter registry. An extension should solve a concrete checkpoint need without weakening the guarantees for existing ones.

Check your understanding

Is “remove the history validation check” a complete adapter?

Answer

No. It removes the failure signal without supplying the history and execution feedback the model actually needs.

23. Source excerpts - all tabs expanded

1. PreTrainedPolicy.chunk_inference_spec

src/lerobot/policies/pretrained.py:307

Excerpt lines 307-338 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

307     def chunk_inference_spec(self) -> ChunkPolicySpec:
308         """Declare the default current-observation chunk-serving author contract.
309
310         Inheriting this implementation declares that ``predict_action_chunk`` does
311         every preparation needed by direct callers, accepts the complete current
312         observation, and returns ``[B, chunk_size, A]`` in model coordinates. Its
313         canonical processor pair must support whole chunks. ``n_action_steps`` is
314         the ordinary playback slice; RTC retains the full prediction horizon.
315
316         A policy requiring select_action-only preparation, other action layouts,
317         or temporal sampling must override this method and reject unsupported
318         configurations or provide a custom runner. Method presence or a successful
319         warmup is not evidence for this contract. Policy/processor state is allowed
320         only within an exclusive session and reset() must clear it for a new run.
321         """
322         config = self.config
323         if config.n_obs_steps != 1:
324             raise ValueError("Chunk serving requires n_obs_steps=1; observation history is u
325 nsupported.")
326         for field_name in (
327             "observation_delta_indices",
328             "image_observation_delta_indices",
329             "state_observation_delta_indices",
330         ):
331             indices = getattr(config, field_name, None)
332             if indices is not None and list(indices) != [0]:
333                 raise ValueError(f"Chunk serving cannot sample temporal history ({field_name
334 }).")
335             if getattr(config, "temporal_ensemble_coeff", None) is not None:
336                 raise ValueError("Chunk serving does not support temporal ensembling.")
337             if getattr(config, "use_visual_memory", False) or getattr(config, "use_proprioceptiv
338 e_memory", False):
339                 raise ValueError(
340                     "Chunk serving cannot sample observation memory at the required control cade
341 nce."
342 )

```

2. LaWAMPolicy.chunk_inference_spec

src/lerobot/policies/lawam/modeling_lawam.py:221

Excerpt lines 221-232. Line numbers refer to the documented source snapshot.

```

221     def chunk_inference_spec(self) -> ChunkPolicySpec:
222         """Serve current observations and the wrapper's already-cropped action horizon.
223
224         Future image indices are training targets for the LAM teacher. Inference
225         predicts the future representation from the current images instead.
226         """
227         if self.config.n_obs_steps != 1:
228             raise ValueError("LaWAM chunk serving requires n_obs_steps=1.")
229         return ChunkPolicySpec(
230             prediction_steps=self.config.action_horizon,
231             execution_steps=self.config.n_action_steps,
232         )

```

3. SmolVLAPolicy.validate_chunk_input_features

src/lerobot/policies/smolvla/modeling_smolvla.py:338

Excerpt lines 338-365. Line numbers refer to the documented source snapshot.

```

338     def validate_chunk_input_features(self, features: tuple[FeatureSpec, ...]) -> None:
339         """Preserve prepare_images' camera subset, resizing and padding semantics.
340
341         The deployment explicitly selects physical cameras. Missing visual slots
342         remain missing; prepare_images applies the checkpoint's empty-camera limit
343         and masks exactly as in local inference. Never synthesize unmasked cameras.
344         """
345         expected = self.config.input_features or {}
346         supplied = {feature.name: feature for feature in features}
347         required = {name for name, feature in expected.items() if feature.type != FeatureType
e.VISUAL}
348         if not required <= supplied.keys() or not supplied.keys() <= expected.keys():
349             raise ValueError("SmolVLA requires all nonvisual inputs and only known camera fe
atures.")
350         if not any(expected[name].type == FeatureType.VISUAL for name in supplied):
351             raise ValueError("SmolVLA requires at least one camera.")
352         for name, feature in supplied.items():
353             policy_feature = expected[name]
354             if policy_feature.type == FeatureType.VISUAL:
355                 shape = policy_feature.shape
356                 if len(shape) != 3 or shape[0] != 3 or feature.kind != "rgb":
357                     raise ValueError(f"SmolVLA requires RGB camera inputs: {name}.")
358                 if self.config.resize_imgs_with_padding is None and feature.shape != (
359                     shape[1],
360                     shape[2],
361                     shape[0],
362                 ):
363                     raise ValueError(f"RGB feature shape differs without policy resizing: {n
ame}.")
364             elif feature.kind != "tensor" or feature.shape != tuple(policy_feature.shape):
365                 raise ValueError(f"Tensor feature shape differs from checkpoint: {name}.")

```

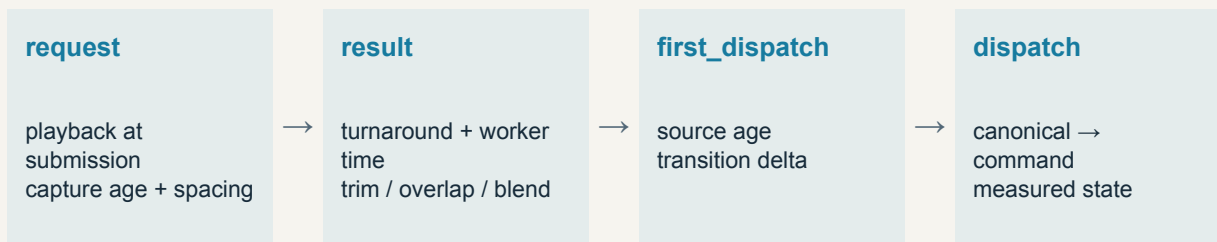
5 · Prepare your review

24. Read logs as a causal chain

5 · PREPARE YOUR REVIEW

Read logs as a causal chain

Match request identity and control ticks before attributing a visible bump.



Server policy time is only one part of client turnaround.

24

Use `configured_refill_s` versus `effective_refill_s` to see whether the measured floor dominates. Compare `playback_at_submission_s` with `turnaround_s` for margin, then inspect accepted status, usable suffix and source age.

A result log can show low network/compute delay while append still delays first dispatch behind old playback. Alignment can reduce that wait while introducing a trajectory discontinuity. `transition_delta` and `replacement_delta` help locate the boundary, but do not independently prove the cause of a physical bump.

When dataset recording is configured, the bounded asynchronous JSONL sidecar records request/result/dispatch provenance. Canonical values, robot-processed commands and measured scalar positions remain distinct. `first_dispatch` is captured at the first queue pop for a request: endpoint commitment, not confirmed hardware arrival or send. `dispatch` records the send path. The bounded sidecar can drop overflowing telemetry, reported at writer close. Source/video correlation needs actual timing; coarse logs alone cannot prove an exact physical cause.

Check your understanding

A server forward takes 100 ms. Is that the latency value used by refill?

Answer

No. The runtime measures complete client turnaround, including transport, server work and response handling. Its steady rolling maximum feeds the refill floor.

24. Source excerpts - all tabs expanded

1. RemoteInferenceEngine._loop

src/lerobot/rollout/inference/remote.py:374

Excerpt lines 462-493 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

462         capture_time=observation.capture_time,
463         task_version=observation.task_version,
464         delay=request.delay,
465         chunk_merge=self.config.chunk_merge,
466         configured_refill_s=self.runtime.refill_seconds,
467         effective_refill_s=self.runtime.effective_refill,
468         scheduling="playback_threshold",
469         request_spacing_s=request_spacing,
470         committed_actions_since_request=committed_since_request,
471         task_changed_since_request=task_changed,
472         recent_turnaround_s=self.runtime.turnaround,
473         playback_at_submission_s=request.playback_at_submission,
474         observation_age_at_submission_s=request.submitted_at - observation.capture
re_time,
475         observation_cursor=observation.action_cursor,
476         submission_cursor=request.continuation.cursor,
477     )
478     self._last_request = (
479         request.generation,
480         request.submitted_at,
481         request.continuation.cursor,
482         observation.task_version,
483     )
484     try:
485         request_generation = request.generation
486
487         def request_cancelled(request_generation: int = request_generation) -> b
ool:
488             return self._stop_event.is_set() or self.runtime.generation != requ
st_generation
489
490         result = self.client.infer(
491             request,
492             cancelled=request_cancelled,
493         )

```

2. RemoteInferenceEngine.record_dispatch

src/lerobot/rollout/inference/remote.py:527

Excerpt lines 527-544. Line numbers refer to the documented source snapshot.

```

527     def record_dispatch(self, canonical: dict, command: dict, measured: dict) -> None:
528         super().record_dispatch(canonical, command, measured)
529         provenance = self.runtime.current
530         if provenance is None:
531             return
532         scalar_state = {
533             key: float(value)
534             for key, value in measured.items()
535             if isinstance(value, (float, int, np.number)) and np.isfinite(value)
536         }
537         self._event(
538             "dispatch",
539             provenance=asdict(provenance),
540             canonical=canonical,
541             command=command,
542             measured=scalar_state,
543             source_age_s=time.monotonic() - provenance.capture_time,
544         )

```

3. ChunkRuntime.pop

src/lerobot/inference/execution.py:445

Excerpt lines 445-476 (method continues in the original file). Line numbers refer to the documented source snapshot.

[illegible]

5 · Prepare your review

25. Tests encode the important counterexamples

5 · PREPARE YOUR REVIEW

Tests encode the important counterexamples

Read the test first when an ordering guarantee feels abstract.

execution / aligned_execution	Capture before submission, committed trimming, timing gates, oldest blend age.
remote_inference / session + engine	Duplicate requests, blocked resets, presence cleanup, retarget and language races.
test_remote_rollout	Interpolation revocation and hold → stop → return → disconnect ordering.
remote path / codec / transport	Checkpoint boundary, malformed payloads and actual process/Zenoh exchange.

Software tests establish bounded cases. Hardware reports establish observed task and motion behavior.

25

Start with `test_observation_anchor_accounts_for_progress_before_and_during_request`: a tiny clock advance demonstrates that plain alignment uses cursor progress, including before submission. Then read the test proving refill waits without reserving an old capture.

For lifecycle, read the blocked-reset and blocked-absence tests: they make model ownership visible by pausing a policy call. For the robot path, read the test that revokes permission during a pull and the test checking terminal shutdown order.

The source excerpts here are documentation evidence; creating this learning guide does not imply those runtime tests were rerun today. Controlled model/robot tests do not replace a real language checkpoint or a representative WAN experiment.

Check your understanding

Which counterexample should fail if alignment accidentally switches to elapsed-time trimming?

Answer

A capture followed by multiple cursor commitments but only a tiny clock advance. The existing anchor test covers exactly that distinction.

25. Source excerpts - all tabs expanded

1. test_observation_anchor_accounts_for_progress_before_and_during_request

tests/inference/test_aligned_execution.py:59

Excerpt lines 59-73. Line numbers refer to the documented source snapshot.

```

59 def test_observation_anchor_accounts_for_progress_before_and_during_request():
60     rt, clock = runtime()
61     initial(rt, clock)
62     rt.pop() # committed endpoint 0, still potentially interpolating toward it
63     clock.now += 0.1
64     source = sample(rt, clock) # action zero assigned to endpoint 1
65     rt.pop() # endpoint 1 committed before request submission
66     request = rt.begin(source)
67     assert request.continuation.cursor == 2
68     rt.pop() # endpoint 2 committed during inference
69     clock.now += 0.001 # wall-time estimate is irrelevant to aligned trimming
70     assert rt.accept(request, result(request, 10), task_version=0)
71     assert rt.last_accept["trimmed_actions"] == 2
72     assert rt.queue.qsize() == 4
73     assert rt.pop()[0].tolist() == [12, 112]

```

2. test_reset_acknowledges_only_after_current_model_call_and_reset

tests/remote_inference/test_session.py:175

Excerpt lines 175-197. Line numbers refer to the documented source snapshot.

```

175 def test_reset_acknowledges_only_after_current_model_call_and_reset(worker):
176     session = admit(worker)
177     entered, release = block_predict(worker)
178     try:
179         action = worker.submit(action_request(worker, session))
180         assert entered.wait(2)
181         reset_request = control_request(worker, session, 1, "reset")
182         reset = worker.submit(reset_request)
183         assert worker.submit(reset_request) is reset
184         assert not reset.done()
185         assert worker.runner.policy.resets == 2
186         assert_error(worker.submit(action_request(worker, session, generation=1)).result(2),
187                      ErrorCode.STALE)
187         release.set()
188         assert action.result(2).message_type is MessageType.ACTION
189         assert reset.result(2).body["applied_generation"] == 1
190         assert worker.runner.policy.resets == 3
191         assert_error(worker.submit(action_request(worker, session)).result(2), ErrorCode.STALE)
192     except LE:
193         assert (
194             worker.submit(action_request(worker, session, generation=1)).result(2).message_type
195             is MessageType.ACTION
196         )
196     finally:
197         release.set()

```

3. test_permission_revoked_during_pull_cannot_dispatch_returned_action

tests/test_remote_rollout.py:117

Excerpt lines 117-122. Line numbers refer to the documented source snapshot.

```

117 def test_permission_revoked_during_pull_cannot_dispatch_returned_action():
118     engine = GateEngine()
119     engine.stop_during_pull = True
120     ctx, robot = make_dispatch_context(engine)
121     assert send_next_action({"joint.pos": 7.0}, {}, ctx, ActionInterpolator()) is None
122     assert robot.sent == [{"joint.pos": 7.0}]

```

5 · Prepare your review

26. A focused first code review

5 · PREPARE YOUR REVIEW

A focused first code review

Follow one guarantee all the way through its implementation and test.

Pass 1

Types + execution

contracts.py →
ChunkRuntime →
ActionQueue

Pass 2

One real request

RemoteInferenceEngine
→ RemoteClient →
SessionWorker →
PolicyRunner

Pass 3

Revoke permission

reset / hold / fault →
interpolation gate →
teardown

Pass 4

Try to break a guarantee

Retarget during
inference; reset during
a blocked model call

Ask: who owns this state, what makes it stale, and which test proves the boundary?

26

For the first review session, choose plain aligned ACT and ignore optional language/RTC internals until you can trace capture → request → merge → dispatch. Keep the policy execution slice separate from the prediction horizon throughout.

For each mutation, note the thread, lock and identity it depends on. For each cached value, ask whether its source timestamp and generation survive. For each callback, check whether it can block or mutate policy state.

Useful review targets are concrete: acceptance racing with task changes; snapshot consistency; a result arriving after invalidation; a blended action nearing its oldest-source age bound; hardware cleanup after a terminal remote fault. Future multi-client serving, cloud packaging and MPC are not prerequisites for reviewing this implementation.

Check your understanding

Can you explain the guarantee behind `runtime.pending` is request?

Answer

It binds acceptance to the exact locally reserved request. A response for a canceled/replaced reservation cannot refill motion merely because some IDs or timestamps look plausible.

26. Source excerpts - all tabs expanded

1. ChunkRuntime.accept

src/lerobot/inference/execution.py:263

Excerpt lines 263-294 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

263     def accept(self, request: ChunkRequest, chunk: ActionChunk, *, task_version: int) -> boo
l:
264         """Validate context and timing, then atomically merge with original provenance."""
265         with self.lock:
266             self.last_accept = {"accepted": False}
267             self.check_deadlines()
268             if self.failure or self.pending is not request or request.generation != self.gen
eration:
269                 return False
270             self.pending = None
271             if request.observation.task_version != task_version:
272                 return False
273             now = self.clock()
274             elapsed = now - request.submitted_at
275             self._completed += 1
276             if self._completed > 1: # cold start is not a steady-state action latency
277                 self.turnarounds.append(elapsed)
278             if now - request.observation.capture_time > self.max_age:
279                 self.fault("Action result source observation is too old")
280                 return False
281             actions = chunk.canonical_actions
282             model = chunk.model_actions if chunk.model_actions is not None else actions
283             if (
284                 actions.ndim != 2
285                 or model.ndim != 2
286                 or len(actions) != len(model)
287                 or not len(actions)
288                 or type(chunk.execution_steps) is not int
289                 or not 0 < chunk.execution_steps <= len(actions)
290                 or not torch.isfinite(actions).all()
291                 or not torch.isfinite(model).all()
292             ):
293                 self.fault("Invalid action chunk")
294                 return False

```

2. RemoteInferenceEngine._loop

src/lerobot/rollout/inference/remote.py:374

Excerpt lines 495-521. Line numbers refer to the documented source snapshot.

```

495         continue
496         # Acceptance and an operator retarget must have one order: an
497         # old in-flight result cannot pass using a previously read version.
498         with self._task_lock:
499             accepted = self.runtime.accept(request, result, task_version=self._task_
version)
500             self._warn_refill_horizon()
501             self._event(
502                 "result",
503                 request_id=request.request_id,
504                 accepted=accepted,
505                 turnaround_s=time.monotonic() - request.submitted_at,
506                 source_age_s=time.monotonic() - observation.capture_time,
507                 queue_playback_s=self.runtime.queue.qsize() * self.runtime.interval,
508                 server_durations=result.server_durations,
509                 merge=self.runtime.last_accept.copy(),
510             )
511         except Exception as exc:
512             self._traceback = traceback.format_exc()
513             self._fault(str(exc))

```

```

514         logger.exception("Remote inference worker fault")
515     finally:
516         if self.runtime.failure is not None:
517             self._fault(self.runtime.failure)
518     try:
519         self.client.close()
520     except Exception:
521         logger.exception("Remote session close failed; presence/idle cleanup will re
lease it")

```

3. send_next_action

src/lerobot/rollout/strategies/core.py:443

Excerpt lines 443-474 (method continues in the original file). Line numbers refer to the documented source snapshot.

```

443 def send_next_action(
444     obs_processed: dict,
445     obs_raw: dict,
446     ctx: RolloutContext,
447     interpolator: ActionInterpolator,
448     timer: CycleTimer | None = None,
449 ) -> dict | None:
450     """Dispatch the next action to the robot.
451
452     Pulls the next action tensor from the inference engine, feeds the
453     interpolator, and sends the interpolated action through the
454     ``robot_action_processor`` to the robot. Works identically for
455     sync and async backends – the rollout strategy never needs to branch.
456
457     When *timer* is given, the engine pull and the robot send are timed as the
458     ``infer`` and ``send`` steps of its cadence summary, and a tick with no action
459     to send is counted there. Note that on async backends ``infer`` is only a
460     queue pull – inference runs off-thread, so its latency surfaces as starved
461     ticks rather than as loop-body time.
462
463     Returns the action dict that was sent, or ``None`` if no action was
464     ready (e.g. empty async queue, interpolator not yet primed).
465     """
466     engine = ctx.policy.inference
467     engine.begin_control_tick()
468     features = ctx.data.dataset_features
469     ordered_keys = ctx.data.ordered_action_keys
470     # ``nullcontext`` accepts (and ignores) the section name, so it stands in for
471     # ``timer.section`` verbatim when no timer was passed.
472     section = timer.section if timer is not None else contextlib.nullcontext
473
474     def dispatch_permitted() -> bool:

```